

Nach welchen Prinzipien lässt sich mit einer LED weißes Licht erzeugen?



Abb. 0.1: Meme: Finally, my time has come¹⁴

Auswertung Versuch234 - Gitterspektrometer

Physikalisches Praktikum PAP 2.2

des Studienganges Physik
an der Universität Heidelberg

von

Benjamin Kleijn

12. August 2026

Versuchstermin 17.02.2026

Tutor:in Pham Huy Thang Le

Inhaltsverzeichnis

1. Einleitung	4
1.1. Motivation	4
1.2. Ziele des Versuches	5
1.3. Geräte	5
1.4. Theoretische und technische Grundlagen	6
1.4.1. Temperaturstrahler	6
1.4.2. Nichttemperaturstrahler	7
1.4.3. Natriumdampflampe	7
2. Durchführung	9
2.1. Messverfahren	9
2.2. Messprotoll	9
3. Auswertung	10
3.1. Sonnenspektrum	12
3.1.1. Absorption der Glasscheibe	12
3.1.2. Fraunhoferlinien	13
3.2. verschiedene Leuchtmittel	15
3.3. Natriumdampflampe	16
3.3.1. plotten und bestimmen der Emissionslinien	16
3.3.2. Berechnung der 1. Nebenserie	17
3.3.3. Berechnung der 2. Nebenserie	17
3.3.4. Berechnung der Hauptserie	18
3.3.5. Zuordnung der gemessenen zu berechneten Linien	19
3.3.6. Nutzen der Messergebnisse zur Bestimmung der Serienenergien und Korrekturfaktoren	20
4. Diskussion	23
4.1. Diskussion	23
4.2. Fazit	24
Literaturquellen	25
Abbildungsquellen	25
A. Rayleigh-Streuung	25
B. Code-Anhang	25

Abbildungsverzeichnis

0.1. <i>Meme</i> : Finally, my time has come ¹⁴	1
1.4. format=plain	8
2.1. These files have been redacted	9
4.1. my aiming skills und gaming-mouse finally paying off ⁵	24



Tabellenverzeichnis

3.1. Vergleich der Fraunhoferlinien in Abb. 3.3 und Abb. 3.4 mit den Literaturwerten	14
3.2. die mittleren Wellenlängen der verschiedenen Lichtquelle	15
3.3. Theoretisch berechnete Emissionslinien der Hauptserie gegenübergestellt mit den gemessenen Wellenlängen	19
3.4. Vergleich der Fitergebnisse mit Literaturwerten/berechneten Größen	21
3.5. Vergleich der Fitergebnisse mit Literaturwerten/berechneten Größen	22

1. Einleitung

1.1. Motivation

Betrachtet man die Fragen, die zur Einleitung und Vorbereitung des Versuches gestellt werden, so wird klar, welch große Bedeutung dieser Versuch und die zugrundeliegenden physikalischen Konzepte haben, z.B.: Optik der Atmosphäre (Streuung, Absorption, Reflektion), Lichtspektrum der Sonne und anderer Lichtquellen. Hinter diesen Überbegriffen steckt eine große Bandbreite von Vorlesungen: Zur Untersuchung der Sonne als Lichtquelle nutzt man das Modell des Schwarzen Strahlers aus der Thermodynamik, optische Phänomene wie Streuung werden in der Experimentalphysik behandelt, Absorption elektro-magnetischer Strahlung durch (Gas-) Atome wird nun mit neu erlerntem Wissen aus der Quantenmechanik (Ex3) vertieft zum Verständnis des Versuches genutzt. Des weiteren setzt der Versuch Impulse, sich mit sehr spannenden Fragen zu beschäftigen: Wie funktionieren LED-Lichter? Warum flackert eine alte [Leuchtstoffröhre](#) beim Einschalten? Warum ist der Himmel tagsüber und während der [blauen Stunde](#) blau?



Abb. 1.1: blaue Stunde im KIP. Wenigstens einmal in der Woche gute Aussicht beim Auswertung schreiben gehabt. Zwar ist die Häuserschlucht da, aber nicht perfekt mit der Sonne aligned, also kein Manhattanhenge für uns.

1.2. Ziele des Versuches

Mithilfe eines Gitterspektrometers wird Sonnenlicht zur Aufzeichnung des Sonnenspektrums untersucht, sodass folgend die Bestimmung dessen Fraunhoferlinien und des absorbierenden Effektes einer Glasscheibe vorgenommen werden kann. Des weiteren wird das Spektrum von künstlichen Lichtquellen, also unterschiedlichen Erzeugungsarten aufgenommen, speziell das Licht der Natriumdampfampe wird dann genauer analysiert, um die theoretisch nach Regeln der Atomphysik erwarteten Wellenlängen mit den gemessenen zu vergleichen.

1.3. Geräte

1. Gitterspektrometer Ocean Optics USB4000 mit Glasfaser
 2. PC mit Auswertungsprogramm OceanView
 3. Natriumdampfampe mit Vorschaltgerät
- *nicht auf dem Bild:* verschiedene Lichtquellen (farbige LEDs, LASER, Energiesparlampe, Glühbirne)

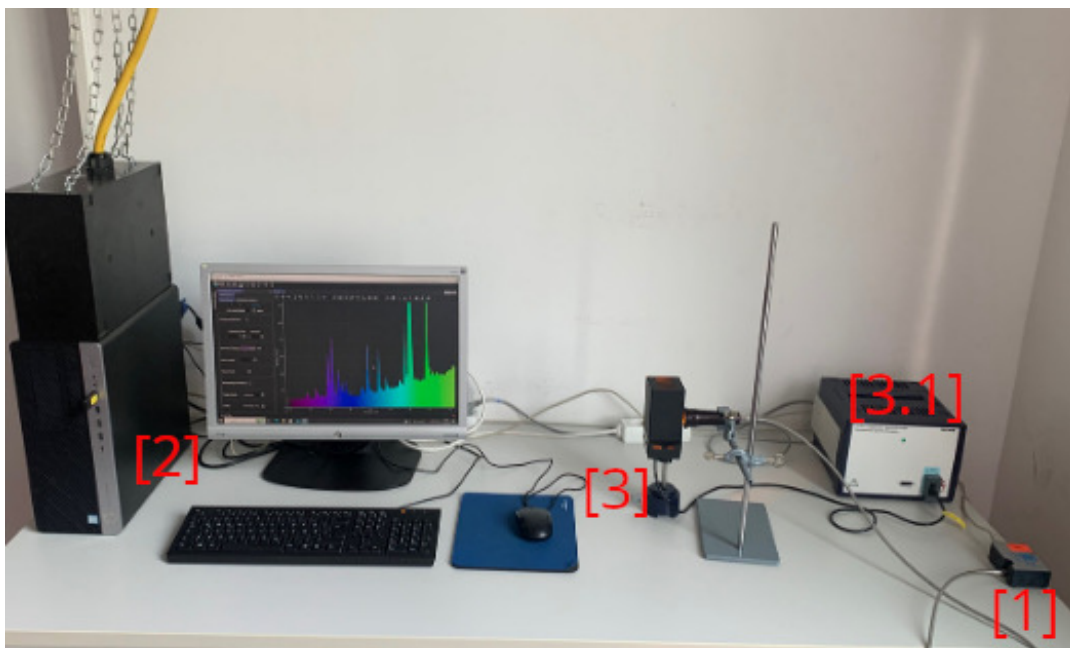


Abb. 1.2: Versuchsaufbau/Geräte²

1.4. Theoretische und technische Grundlagen²

Grundsätzlich kann man die Emission von elektromagnetischer Strahlung in zwei verschiedene Klassen teilen: Temperaturstrahler senden Licht eines kontinuierlichen Wellenlängenspektrums aus, diskrete Lichtquellen (Nichttemperaturstrahler) hingegen emittieren nur bestimmte Wellenlängen.

1.4.1. Temperaturstrahler

Nach den Modellen der Thermodynamik emittiert jeder Körper mit Temperatur $T > 0\text{ K}$ elektromagnetische Strahlung. Beim Modell des schwarzen Strahlers, welcher alle eintreffende Strahlung absorbiert und gleichzeitig das maximal mögliche Emissionsvermögen $\epsilon = 1$ besitzt, lässt sich das Plancksche Strahlungsgesetz aufstellen, welches die Strahlungsleistung M_λ des Flächenelements dA innerhalb des Wellenlängenbereichs $\lambda + d\lambda$ bei Körpertemperatur T angibt:

$$M_\lambda(\lambda, T) dA d\lambda = \frac{2\pi hc^2}{\lambda^5 \exp(\frac{hc}{\lambda kT}) - 1} dA d\lambda \quad (1.1)$$

Der Stern Sonne lässt sich als schwarzer Strahler annähern: Mit der Temperatur $T = 5777\text{ K}$ ergibt sich die Strahlungsleistung M_λ wie in Abb. 1.3.

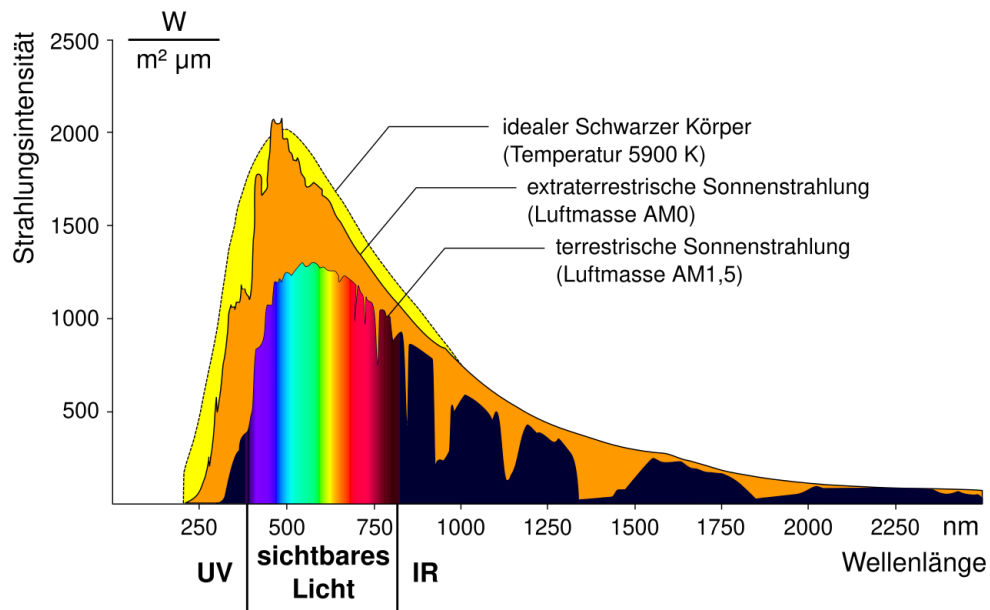


Abb. 1.3: Intensität der Sonnenstrahlung mit und ohne Erdatmosphäre sowie erwartbare Verteilung nach dem Planckschen Strahlungsgesetz, Gleichung (1.1)⁶

Die Suche nach dem eindeutig bestimmbar Maximum von M_λ kann durch ein numerisches

Verfahren gelöst werden, welches auf das Wiensche Verschiebungsgesetz führt:

$$\lambda_{\max} T = \frac{2897.8 \mu\text{m K}}{T} \quad (1.2)$$

Dies bedeutet, dass ein Körper mit einer höheren Temperatur mehr Energie in Form von Strahlung abgibt und diese Strahlung eine kürzere Wellenlänge (bei Größenordnung von sichtbarem Licht: Blauverschiebung des Maximums) hat. Dieses temperaturabhängige Spektrum und sein durchs Wiensche Verschiebungsgesetz bestimmtes Maximum sorgen für die Kennzeichnung einer Lichtquelle nach ihrer Farbtemperatur: Bei niedrigen Temperaturen dominieren die Rotanteile des sichtbaren Lichts im Spektrum gegenüber den anderen Lichtfarben im sichtbaren Bereich, für höhere Temperaturen dominieren die Blauanteile.

In Abb. 1.3 sind noch andere Dinge erkennbar: Sowohl bei der extraterrestrischen, als auch der terrestrischen Strahlung sind an manchen Wellenlängen Absorptionslinien zu sehen. Stoffe/Atome in der Sonnen- bzw. Erdatmosphäre absorbieren bestimmte Wellenlängen, und emittieren die Energie der absorbierten Atome dann spontan in eine zufällige Richtung.

Für die Beobachtung des Sonnenlichts am Erdboden spielt noch ein weitere Effekt eine große Rolle: Die Rayleigh-Streuung. Durch die wellenlängen-abhängige Streuung von Sonnenlicht an Atomen der Atmosphäre wird blaues Licht in alle möglichen Richtungen gestreut, die Summe allen Streulichts dominiert dadurch die Farbe des Himmels - blau. In der Morgen- und Abenddämmerung wird durch einen längeren Atmosphärenweg ein Großteil des blauen Lichts seitlich herausgestreut und trifft dadurch nicht das Auge des Betrachters - der Himmel erscheint rötlich.

1.4.2. Nichttemperaturstrahler

Das Licht von Nichttemperaturstrahlern baut entweder auf dem Übergang eines Elektrons zwischen zwei Atomzuständen in Gasen oder Festkörpern, der bei Relaxation (Elektron fällt auf einen niedrigen Zustand zurück) die Differenzenergie der Zustände als Photon aussendet oder auf Rekombination von Elektron-Loch Paaren in Halbleitern.

Die Energieniveaus eines Atoms sind diskret, somit ist auch die pro Photon emittierte Energie ($E_{ph} = \frac{hc}{\lambda}$) diskret, dies ist auch beim [pn-Übergang](#) in Halbleitern der Fall, hier geht ein Elektron von der n-dotierten in die p-dotierte Schicht über, wenn eine Spannung an der Diode angelegt wurde. Im Banddiagramm entspricht das dem Übergang vom Leitungsband in das Valenzband des Halbleiters. Deren energetischer Abstand, der Bandgap, entscheidet die Energie des ausgesendeten Photons.

1.4.3. Natriumdampfampe

Im Periodensystem lässt sich ablesen, dass Natrium die Ordnungszahl $z = 11$ hat und ein einzelnes Außenelektron in der $n = 3$ - Hauptschale besitzt. In Abhängigkeit seines Drehim-

pulses kann dieses Elektron nun verschiedene Energiezustände annehmen, welche neben der Hauptquantenzahl n durch den quantisierten Drehimpuls und der zugehörigen Quantenzahl l bestimmt werden, es wurde also die 1-Entartung aufgehoben - die Auswahlregel für Übergänge in wasserstoffähnlichen Atomen $\Delta n = 1$ gilt nicht mehr. Die Energie des Elektrons berechnet sich bei vorliegendem Potential zu:

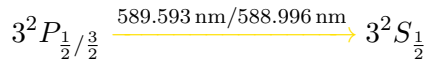
$$E_{n,l} = -13.6 \text{ eV} \frac{1}{(n - \Delta_{n,l})^2} \quad (1.3)$$

Es sind nun eine Vielzahl an Übergängen möglich: Anhand des **Termsymbols** der Atomphysik und den Eigenschaften des untersuchten Natriums können diese beschrieben werden.

$$n^{2S+1}L_J \left\{ \begin{array}{l} \text{Hauptquantenzahl, für Natrium: } n = 3, 4, 5, 6, \dots \\ \text{Bahndrehimpuls } L = 0, 1, 2, 3, \dots, n \equiv S, P, D, F, \dots \\ \text{Spin } S: S_{\text{Elektron}} = \frac{1}{2} \\ \text{Gesamtdrehimpuls } J = |L - S|, |L - S| + 1, \dots, L + S \end{array} \right.$$

Das Elektron des Natriumatoms wird also durch die Gasentladungslampe angeregt, dies erfolgt mit Hilfe eines elektrischen Feldes welches Elektronen beschleunigt, die ihre Energie dann per Stoßionisation an die Natriumatome weitergeben.

Der bedeutendste Übergang der Relaxation des Elektrons in einer Natriumdampfampe ist das gelbe Dublett vom Zustand:



In Diagramm 1.4 sind neben der Hauptserie ($n^2P_{\frac{1}{2}/\frac{3}{2}} \rightarrow 3^2S_{\frac{1}{2}}$), die 1. ($n^2D_{\frac{1}{2}/\frac{3}{2}} \rightarrow 3^2P$) und 2. Nebenserie ($n^2S_{\frac{1}{2}} \rightarrow 3^2P$) eingezeichnet.

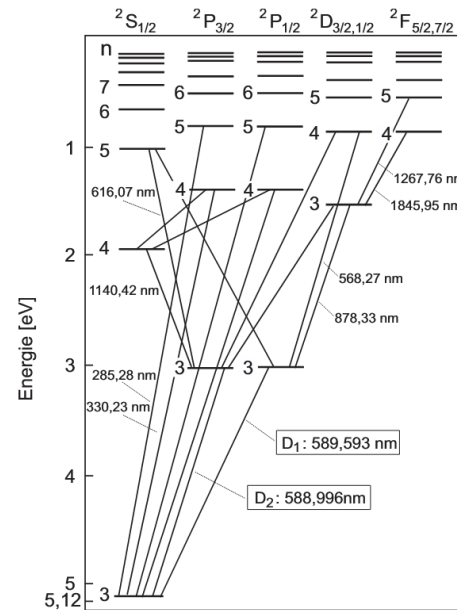


Abb. 1.4: Energieschema und Photonübergänge im Natriumatom (Grotriandiagramm)²

2. Durchführung

2.1. Messverfahren

Für alle Messungen wird ein Gitterspektrometer verwendet, welches das entsprechende Licht über ein Objektiv mit Lichtleitfaser aufnimmt und es durch die dispersiven Eigenschaften eines Gitters so aufspaltet, dass mithilfe eines LCD-Sensors die einzelnen Wellenlängen detektiert werden können. Das resultierende Spektrum (Intensität aufgetragen über Wellenlänge) wird am Computer durch OceanView als Diagramm angezeigt und kann sowohl als `.pdf` als auch als `.txt` Datei abgespeichert werden. Durch Fokussieren und Ausrichten des Objektives auf die jeweilig zu untersuchende Lichtquelle werden die Messergebnisse erhoben.

Neben Sonnenlicht, blauem Himmel vor und hinter Glas, sowie den verschiedenen Lichtquellen, wurde die Natriumlampe genauer untersucht. Dafür wurde das Spektrum im Bereich von (400 bis 540) nm einmal aufgenommen, sodass der Peak bei 500 nm in Sättigung ist (also nicht vollständig auf dem Diagramm zu sehen, da die Skalierung der Intensitätsachse zu klein ist), dadurch können die schwachen Peaks besser anvisiert werden, und einmal nicht mit dem Peak in Sättigung.

In Aufgabe 4 werden nun zwei Linien bei 570 nm und 615 nm sichtbar eingestellt, die sonst von der D-Doppellinie (Dublett) überstrahlt werden, die D-Linie wird auch aufgezeichnet.

Aufgabe 5 verlangt nun noch die Aufzeichnung des Spektrums im Bereich von (650 bis 850) nm.

2.2. Messprotoll

Es liegt kein Messprotokoll vor, die erhobenen Daten wurden in den entsprechenden `.txt` Dateien abgespeichert.

```

1 Data from USB4[REDACTED].txt Node
2
3 Date: [REDACTED]
4 User: [REDACTED]
5 Spectrometer: USB4FCKNZS4EVER
6 Trigger mode: aggression = max
7 [REDACTED]
8 Scans to average: [REDACTED]
9 Redaction dark enabled: true
10 Rage mode enabled: true
11 Number of useless hours in [REDACTED] 2.1: 3648
12 >>[REDACTED] of timewasting<<[REDACTED]
```

Abb. 2.1: These files have been redacted

3. Auswertung

Formeln der statistischen Analyse³

Varianzen: Für die statistische Auswertung von n Messwerten $\{x_i\}_{i=1}^n$ werden folgende Größen definiert:

$$\text{Arithmetisches Mittel} \quad \bar{x} = \text{Mean}(\{x_i\}_{i=1}^n) = \frac{1}{n} \sum_{i=1}^n x_i \quad (\text{S.1})$$

$$\text{Varianz} \quad \sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (\text{S.2})$$

$$\text{Standardabweichung} \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (\text{S.3})$$

$$\text{Fehler des Mittelwerts} \quad \Delta \bar{x} = \frac{\sigma}{\sqrt{n-1}} \quad (\text{S.4})$$

Fehlerfortpflanzung: Der Fehler einer Funktion $f(x_1, \dots, x_N)$, die von den Variablen $\{x_i\}_{i=1}^N$ abhängt, wird wie folgt bestimmt:

$$\text{Gauß'sche Fehlerfortpfl.} \quad \Delta f = \sqrt{\sum_{i=1}^N \left(\frac{\partial f}{\partial x_i} \Delta x_i \right)^2} \quad (\text{S.5})$$

$$\text{relativer Fehler von } f = \prod_{i=1}^N x_i^{\alpha_i} \quad \frac{\Delta f}{|f|} = \sqrt{\sum_{i=1}^N \left(\frac{\alpha_i \Delta x_i}{x_i} \right)^2} \quad (\text{S.6})$$

Ergebnissignifikanz: Resultate eines Experiments bzw. Berechnung a_{exp} und die entsprechenden Literaturwerte a_{lit} werden folgend verglichen:

$$\begin{aligned} \text{Absolute Abweichung} \quad A &= |a_{\text{lit}} - a_{\text{exp}}| \\ \Delta A &= \sqrt{(\Delta a_{\text{lit}})^2 + (\Delta a_{\text{exp}})^2} \end{aligned} \quad (\text{S.7})$$

$$\text{Relative Abweichung} \quad R[\%] = \frac{A}{a_{\text{lit}}} \cdot 100 \quad (\text{S.8})$$

$$\text{Signifikanz} \quad S[\sigma] = \frac{A}{\Delta A} = \frac{|a_{\text{lit}} - a_{\text{exp}}|}{\sqrt{\Delta a_{\text{lit}}^2 + \Delta a_{\text{exp}}^2}} \quad (\text{S.9})$$

Fitgüte: Für einen Fit mit Funktionswerten $\{F_{a_1, \dots, a_m}(x_i)\}_{i=1}^n$ und Fitparametern $\{a_j\}_{j=1}^m$ an die experimentellen Messwerte $\{y_i\}_{i=1}^n$ mit Fehler $\{\Delta y_i\}_{i=1}^n$ kann die Güte des Fits mit diesen Größen analysiert werden:

$$\chi^2 \text{ Summe} \quad \chi^2 = \sum_{i=1}^n \left(\frac{F(x_i) - y_i}{\Delta y_i} \right)^2 \quad (\text{S.10})$$

$$\text{Freiheitsgrade} \quad f = n - m \quad (\text{S.11})$$

$$\text{normierte reduzierte } \chi^2 \text{ Summe} \quad \chi_{\text{red}}^2 = \frac{\chi^2}{f} \quad (\text{S.12})$$

$$\text{Fitwahrscheinlichkeit} \quad p = 1 - \text{chi2.cdf}(\chi^2, f) \quad (\text{S.13})$$

Hierbei wird das Modul `chi2` von `scipy.stats` benutzt.

3.1. Sonnenspektrum

3.1.1. Absorption der Glasscheibe

Folgend ist das Sonnenspektrum mit und ohne Glas zu sehen. Es ist ein erkennbarer Intensitätsunterschied vorhanden.

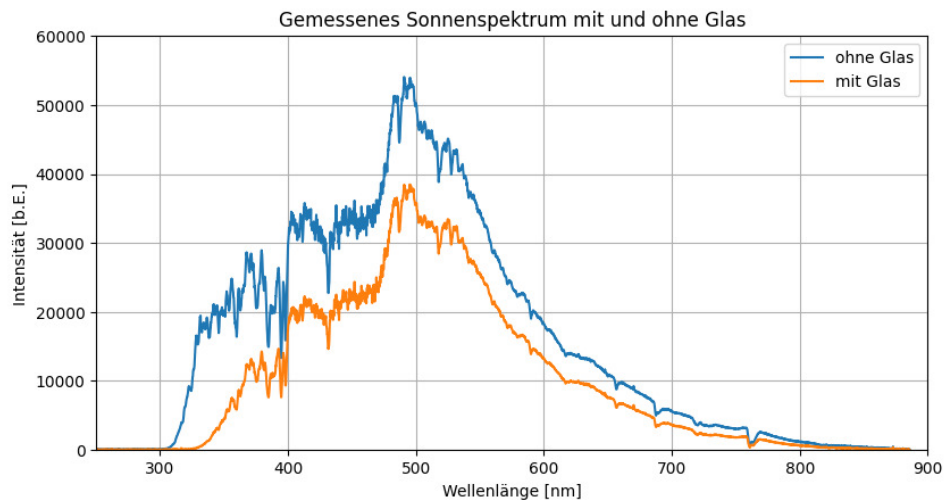


Abb. 3.1: Intensität des Sonnenspektrums im sichtbaren Bereich

Die Berechnung der Absorption wird mithilfe der Formel durchgeführt:

$$A_{\text{Glas}}(\lambda) = 1 - \frac{I_{\text{mG}}(\lambda)}{I_{\text{oG}}(\lambda)} \quad (3.1)$$

Es ergibt sich der Graph:

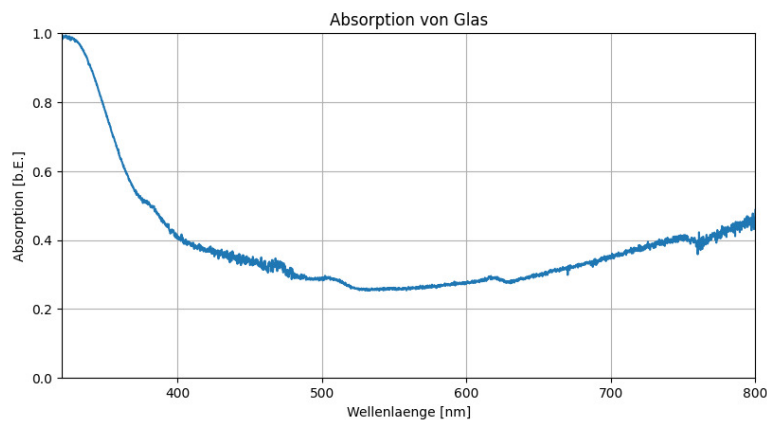


Abb. 3.2: Absorptionsauswirkung eines Glasfensters auf das Sonnenspektrum

Wir erkennen, dass die Absorption von Glas nicht für alle Wellenlängen gleich ist und gerade

im niedrigen Wellenlängenbereich (dem für Menschen schädlichen UV-Licht) am größten ist, während es im sichtbaren Bereich eine relativ geringe Absorption aufweist. Im Infrarotbereich (Wärmestrahlung) steigt die Absorption wieder annähernd linear an, da das Fensterglas ein wärmeisolierendes Fassadenobjekt ist.

3.1.2. Fraunhoferlinien

In Abb. 3.3 ist das Sonnenspektrum zu sehen, in dem anhand der zu sehenden Intensitätsdips schwarze Linien eingezeichnet wurden. In rot ist die Balmerserie des Wasserstoff-Atoms zu sehen, und in golden die Heliumlinie, die Literaturwerte siehe².

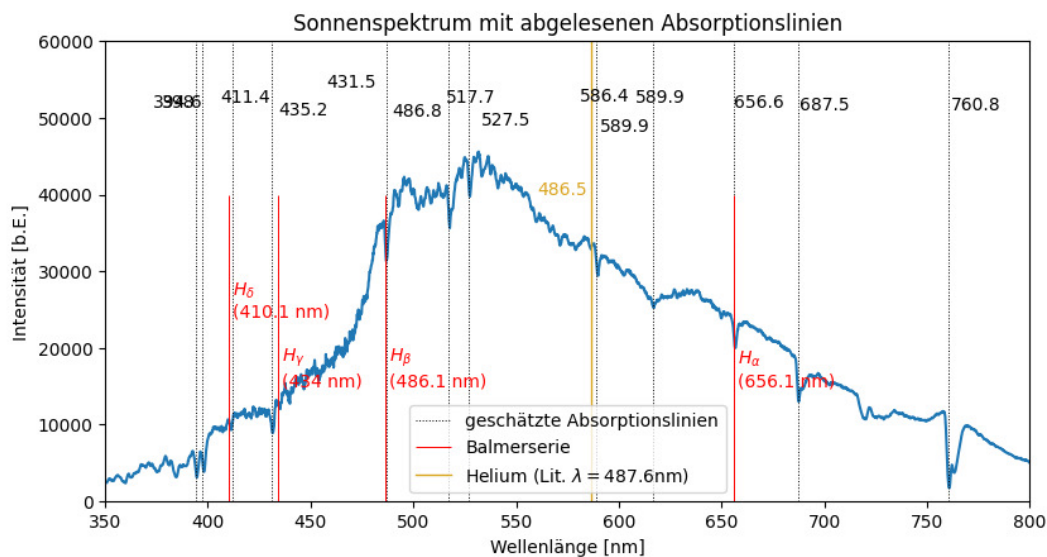


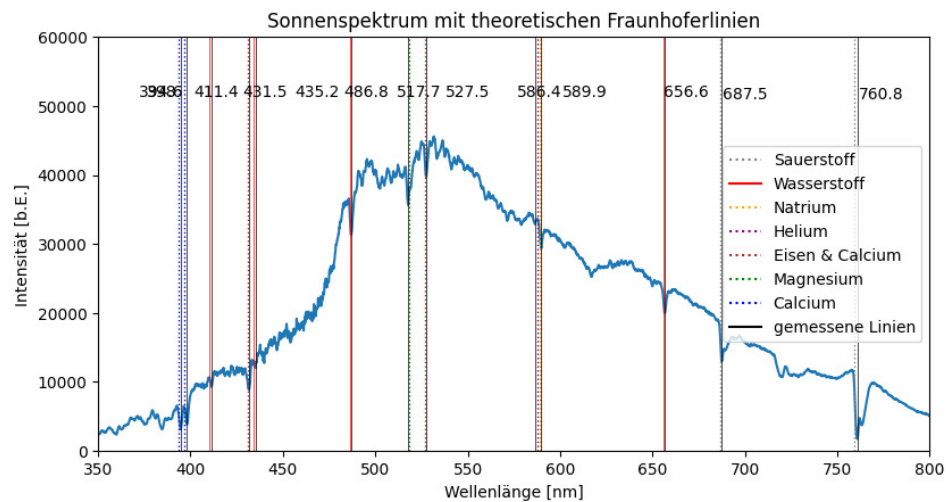
Abb. 3.3: gemessene Fraunhoferlinien und eingezeichnete theoretische Balmerserie

Zur Vermeidung eines überladenen Diagramms ist in Abb. 3.4 das selbige Sonnenspektrum eingezeichnet, nun zusätzlich der theoretischen Fraunhoferlinien von Gasen der Erd- und Sonnenatmosphäre. Nun werden die gemessenen Wellenlängen (deren Position bestmöglich mit dem Cursor erfasst wurde) mit den Literaturwerten verglichen. Als Messfehler dient pauschal $\Delta\lambda = 1 \text{ nm}$.

Tabelle 3.1: Vergleich der Fraunhoferlinien in Abb. 3.3 und Abb. 3.4 mit den Literaturwerten

Element	λ_{theo} [nm]	λ_{exp} [nm]	$\sigma_{Abw.}$
Ca	393.4	394.6 ± 1.0	1.2
	396.8	398.0 ± 1.0	1.2
Fe,Ca	430.8	431.5 ± 1.0	0.7
	527.0	527.5 ± 1.0	0.5
Mg	518.4	517.7 ± 1.0	0.7
He	587.6	586.4 ± 1.0	1.2
Na	589.0	589.9 ± 1.0	0.9
	589.6	589.9 ± 1.0	0.3
Ca	686.7	687.5 ± 1.0	0.8
	759.4	760.8 ± 1.0	1.4
H	410.1	411.4 ± 1.0	1.3
	434.1	435.2 ± 1.0	1.1
	486.1	486.8 ± 1.0	0.7
	656.1	656.6 ± 1.0	0.5

Hier sind nochmal alle Linien eingezeichnet:

**Abb. 3.4:** gemessene Fraunhoferlinien und Literaturwerte einzelner Atmosphärenelemente

3.2. verschiedene Leuchtmittel

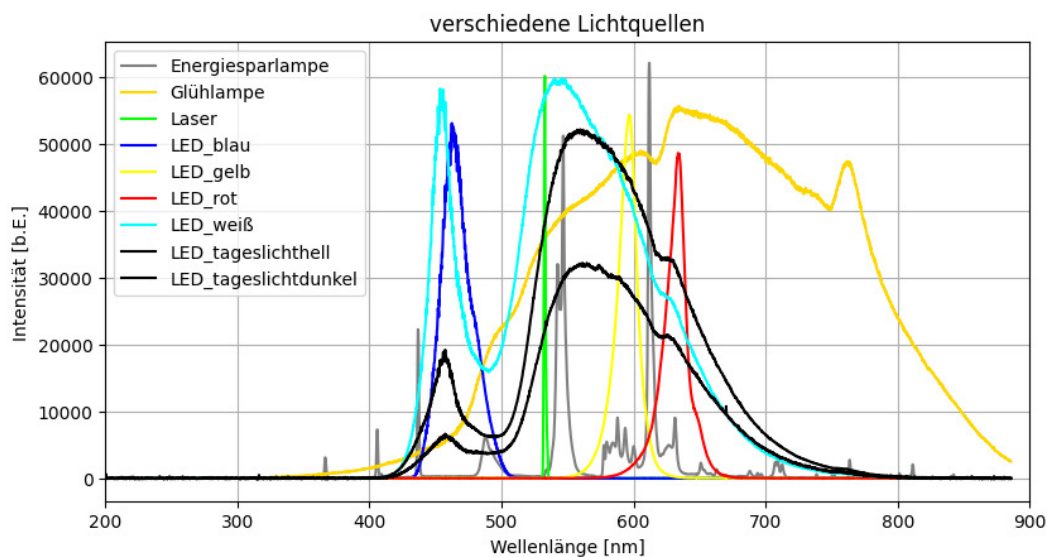


Abb. 3.5: Spektren verschiedener Lichtquellen

Es wurden viele verschiedene Leuchtmittel untersucht, deren Spektren sind in folgendem Diagramm überlagert zu sehen. Die mittleren Wellenlängen lassen sich mithilfe folgender Formel berechnen, die aus einem Altprotokoll kopiert wurde:

$$\bar{\lambda} = \frac{\sum \lambda_i I_i}{\sum I_i} \quad (3.2)$$

Hiermit ergeben sich die folgenden Werte:

Tabelle 3.2: die mittleren Wellenlängen der verschiedenen Lichtquelle

Lichtquelle	$\bar{\lambda}$ [nm]
Energiesparlampe	573.91
Glühlampe	658.13
Laser	534.54
LED blau	467.16
LED gelb	594.17
LED rot	630.93
LED weiß	553.64

Wie erwartet ist die effizienteste Lichtquelle hinsichtlich maximal mögliche Monochromatizität der Laser, gefolgt von den LEDs. Gut erkennbar ist das breite Spektrum der Glühlampe, welche bis in die Infrarotstrahlung übergeht. Deren mittlere Wellenlänge ist rotverschoben,

weswegen ihre Farbtemperatur als warmweiß erscheint. Gut zu sehen sind auch die einzelnen Emissionslinien der Energiesparlampe, einer Gasentladungslampe mit Quecksilberdampf, dessen ultraviolettes Spektrum dann von einem Leuchtstoff an der Innenseite in sichtbares Licht umgewandelt wird. Die mittlere Wellenlänge liegt im grün-gelben Bereich, was die Lampe eher als kaltweiß erscheinen lässt.

Interessant ist auch das Spektrum der weißen LED. Dieses hat einen breiten blauen Peak und einen breiten rötlichen Peak mit hoher Intensität. Die mittlere Wellenlänge ist größer als die der Leuchtstoffröhre, weshalb das Licht nicht mehr so kalt erscheint. Das Spektrum der weißen LED zeigt, dass das hier betrachtete weiße Licht vermutlich nicht durch eine gleichmäßige Überlagerung der Farben rot, grün und blau entsteht, sondern durch ein deutlich breiteres Spektrum im blau-grünen sichtbaren Bereich. Dies steht im Einklang zu einer üblichen Bauweise von weißen LEDs: Eine Lumineszenz-Schicht ist an der Innenseite aufgetragen, die die Wellenlängen des blauen Lichts ins gelbliche verschiebt. Ich vermute, dass der starke blaue Peak von der blauen LED stammt, und der verschobene im grün-gelblichen durch die Fluoreszenzschicht entsteht. Wie erwartet, ist bei den LEDs als energieeffizientester Lichtquelle keine Infrarotstrahlung zu sehen.

3.3. Natriumdampf Lampe

3.3.1. plotten und bestimmen der Emissionslinien

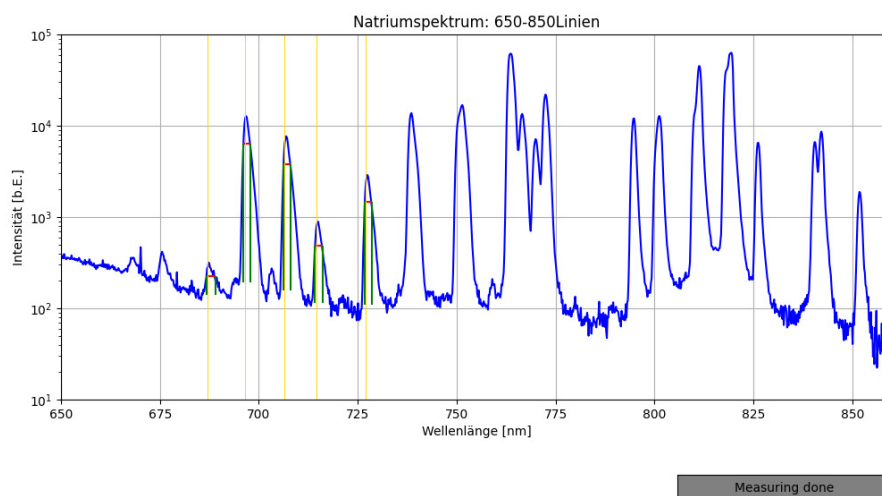


Abb. 3.6: Analyse des logarithmisch aufgetragenen Spektrums von Natrium im Infrarotbereich

Die ganzen restlichen Plots sind im Anhang zu finden, hier ist beispielhaft einer aufgetragen. Zur einfacheren Analyse habe ich mit Unterstützung von ChatGPT eine kleine Funktion geschrieben, die automatisch durch Anklicken der Peaks im Plot Emissionslinien hinzufügt und deren Halbwertsbreite bestimmt. Eigentlich sollte dies die Breite des Peaks bei halber

Maximumsintensität sein, da manche Spektren jedoch ein Grundrauschen/Grundintensität beinhalten, da die Integrationszeit stark erhöht wurde, um schwache Peaks sichtbar zu machen, wird bei jedem Peak eine Referenznullhöhe (am „Fuß“ des Peaks) durch erstmaliges Klicken festgelegt. Durch Drücken des Buttons wird der Prozess gestoppt, der Plot gespeichert und die Daten mit Fehler als Latex-Code in einer `.txt` Datei gespeichert.

Durch Berechnen der theoretischen Werte der Nebenserie und vergleichen mit den in den `.txt` Dateien gespeicherten, gemessenen Wellenlängen, wurde die finale Tabelle 3.3 in Abschnitt 3.3.5 (Zuordnung der Linien) erstellt.

3.3.2. Berechnung der 1. Nebenserie

Als erstes berechnen wir die theoretischen Werte der Linien der 1. Nebenserie $ms \rightarrow 3p$. Hierbei ist die Korrektur nahezu Null, weshalb sich die Gleichung für die Grundenergie E_{3p} zu

$$E_{3p}[\text{eV}] = \frac{E_{Ry}[\text{eV}]}{m^2} - \frac{hc}{\lambda_m} \quad \Delta E_{3p} = \frac{hc}{\lambda_m^2} \Delta \lambda_m \quad (3.3)$$

bestimmt. Hierbei sind die Konstanten h das Plancksche Wirkungsquantum, c die Lichtgeschwindigkeit und $E_{Ry} = -hcR_\infty = -13.605 \text{ eV}$ die Rydbergenergie. Nun ordnet man dem Peak bei $\lambda = (819.0 \pm 1.0) \text{ nm}$ den Wert $m = 3$ zu und berechnet mit Gleichung (3.3) die Energie:

$$E_{3p} = (-3.0256 \pm 0.0028) \text{ eV}$$

Mithilfe dieses Wertes konnten nun die restlichen Werte für λ_m $m \in (3, 13)$ berechnet werden, die in Tabelle 3.3 zu finden sind. Dies geschieht, indem die obige Gleichung nach λ_m umgestellt wird:

$$\lambda_m = \frac{hc}{\left(\frac{E_{Ry}}{m^2} - E_{3p}\right)} \quad \Delta \lambda_m = \frac{hc}{\left(\frac{E_{Ry}}{m^2} - E_{3p}\right)^2} \Delta E_{3p} \quad (3.4)$$

3.3.3. Berechnung der 2. Nebenserie

Unter Zuhilfenahme der eben berechneten Energie E_{3p} kann mithilfe folgender Formeln die zweite Nebenserie $mp \rightarrow 3s$ berechnet werden. Da die D-Linie, also der Übergang $3p \rightarrow 3s$, der Wellenlänge $\lambda = (589.0 \pm 1.0) \text{ nm}$ entspricht, kann so die Grundenergie E_{3s} berechnet werden.

$$E_{3s} = E_{3p} - \frac{hc}{\lambda} \quad \Delta E_{3s} = \sqrt{(\Delta E_{3p})^2 + \left(\frac{hc}{\lambda^2} \Delta \lambda\right)^2} \quad (3.5)$$

$$E_{3s} = (-5.131 \pm 0.006) \text{ eV}$$

Denn diese Grundenergie der 2.Nebenserie wird nun benutzt, um den notwendigen Korrekturfaktor Δ_s zu berechnen:

$$\Delta_s = 3 - \sqrt{\frac{E_{Ry}}{E_{3s}}} \quad \Delta\Delta_s = \frac{1}{2} \sqrt{\frac{E_{Ry}}{E_{3s}^3}} \Delta E_{3s} \quad (3.6)$$

Somit berechnet sich:

$$\Delta_s = 1.3715 \pm 0.0010$$

Schlussendlich lässt sich wieder die gesuchte Formel für λ_m $m \in (4, 9)$ aufstellen:

$$\lambda_m = \frac{hc}{\frac{E_{Ry}}{(m-\Delta_s)^2} - E_{3p}} \quad \Delta\lambda_m = \frac{hc}{\left(\frac{E_{Ry}}{(m-\Delta_s)^2} - E_{3p}\right)^2} \sqrt{\left(\frac{2E_{Ry}}{(m-\Delta_s)^3} \Delta\Delta_s\right)^2 + (\Delta E_{3p})^2} \quad (3.7)$$

3.3.4. Berechnung der Hauptserie

Der Korrekturfaktor der Hauptserie Δ_p bestimmt sich durch:

$$\Delta_p = 3 - \sqrt{\frac{E_{Ry}}{E_{3p}}} \quad \Delta\Delta_p = \frac{1}{2} \sqrt{\frac{E_{Ry}}{E_{3p}^3}} \Delta E_{3s} \quad (3.8)$$

$$\Delta_p = 0.8794 \pm 0.0019$$

Zur Bestimmung der Wellenlängen für $m \in (4, 5)$ wird wieder Gleichung (3.7) benutzt, nur ersetzt man $\Delta_s \rightleftharpoons \Delta_p$ und $E_{3s} \rightleftharpoons E_{3p}$.

3.3.5. Zuordnung der gemessenen zu berechneten Linien

In Tabelle 3.3 wurden alle Ergebnisse der vorherigen Abschnitte zusammengetragen: Die aus den Plots per Hand abgelesenen Emissionslinien, deren Werte dann den anhand der obigen Formeln zu theoretisch berechneten Linien zugeordnet wurden.

Tabelle 3.3: Theoretisch berechnete Emissionslinien der Hauptserie gegenübergestellt mit den gemessenen Wellenlängen

m	λ_{theo} [nm]	λ_{exp} [nm]	$\sigma_{Abw.}$
1.Nebenserie			
3	819.0 ± 1.5	819.1 ± 1.1	0.05
4	570.0 ± 0.7	569.2 ± 1.0	0.66
5	499.7 ± 0.6	498.9 ± 1.0	0.69
6	468.3 ± 0.5	467.6 ± 0.9	0.68
7	451.2 ± 0.5	452.2 ± 1.7	0.56
8	440.8 ± 0.4	439.7 ± 1.1	0.94
9	433.9 ± 0.4	434.7 ± 1.0	0.74
10	429.1 ± 0.4	428.2 ± 1.4	0.62
11	425.6 ± 0.4	—	—
12	423.0 ± 0.4	—	—
13	421.0 ± 0.4	421.1 ± 1.1	0.09
2.Nebenserie			
4	1173.8 ± 3.5	—	—
5	622.4 ± 0.9	616.2 ± 0.4	6.30
6	518.7 ± 0.6	515.8 ± 1.0	2.49
7	477.6 ± 0.5	475.8 ± 1.1	1.49
8	456.5 ± 0.5	456.0 ± 1.0	0.45
9	444.1 ± 0.4	439.7 ± 1.1	3.76
Hauptserie			
4	332.1 ± 0.6	—	—
5	286.4 ± 0.4	—	—

Dass der erste Wert übereinstimmt ist klar (dieser wurde ja in Wahrheit experimentell bestimmt) und auch die geringen Sigma-Abweichungen der 1.Nebenserie sind erfreulich, aber nicht verwunderlich, da immer der am besten passende Werte gewählt wurde. Die Peaks für $m = 11, 12$, bei denen die Zuordnung nicht klar war, wurden z.B. gar nicht erst in die Tabelle aufgenommen.

Der Hauptserie konnte kein Wert zugeordnet werden, da wie in Abbildung Abb. 3.7 erkennbar, der Peak hier in Sättigung ist. Kleinere Wellenlängen als 350 nm wurden nicht vermessen.

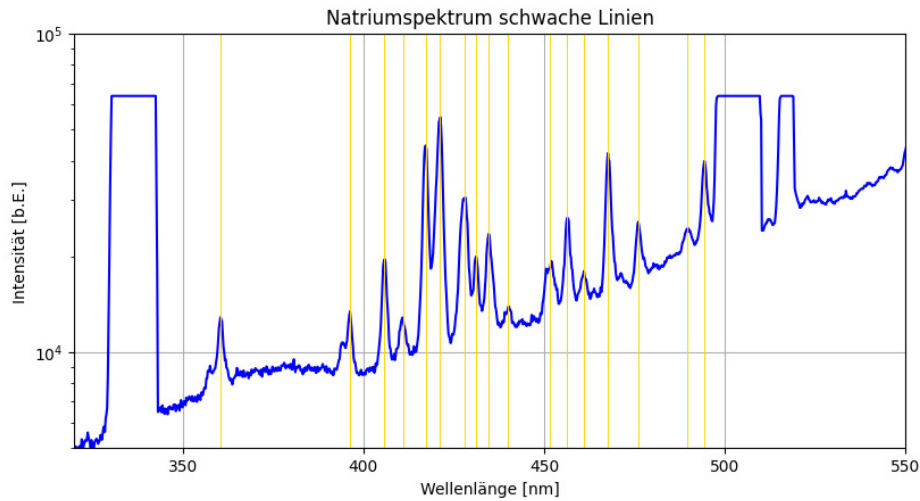


Abb. 3.7: Analyse des logarithmisch aufgetragenen Spektrums von Natrium im kurzwelligen Bereich

3.3.6. Nutzen der Messergebnisse zur Bestimmung der Serienenergien und Korrekturfaktoren

Für die zwei Nebenserien können die einer Quantenzahl m zugeordneten Wellenlängen λ_m nun in einem Diagramm aufgetragen werden, und eine Funktion angefitet werden.

Bei der 1. Nebenserie, wird Gleichung (3.4) genutzt, nur diesmal mit $(m - \Delta_d)^2$ im Nenner:

$$\lambda_m(m, E_{Ry}, E_{3p}, \Delta_d) = \frac{hc}{\left(\frac{E_{Ry}}{(m - \Delta_d)^2} - E_{3p}\right)} \quad (3.9)$$

Δ_d ist der durchs Fitten zu bestimmende Korrekturfaktor, E_{Ry} und E_{3p} sollen ebenso bestimmt werden. Der Fit ergibt folgende Ergebnisse:

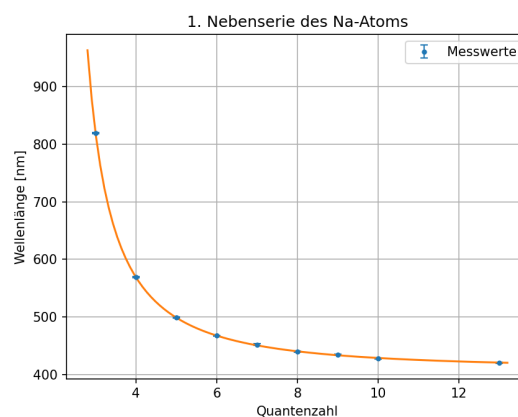


Abb. 3.8: Fitten der Messergebnisse aus Tabelle 3.3 an Funktion 3.9

$$E_{Ry} = (-13.31 \pm 0.23) \text{ eV} \quad E_{3p} = (-3.023 \pm 0.004) \text{ eV}$$

$$\Delta_d = 0.031 \pm 0.023$$

Tabelle 3.4: Vergleich der Fitergebnisse mit Literaturwerten/berechneten Größen

Größe	Vergleichswert	Fit-Wert	σ_{Abw}	proz. Abw. [%]
$E_{Ry}[\text{eV}]$	-13.605 ± 0	-13.31 ± 0.23	1.3	2.2
$E_{3p}[\text{eV}]$	-3.0256 ± 0.0028	-3.023 ± 0.004	0.5	0.09
Δ_d	—	—	—	—

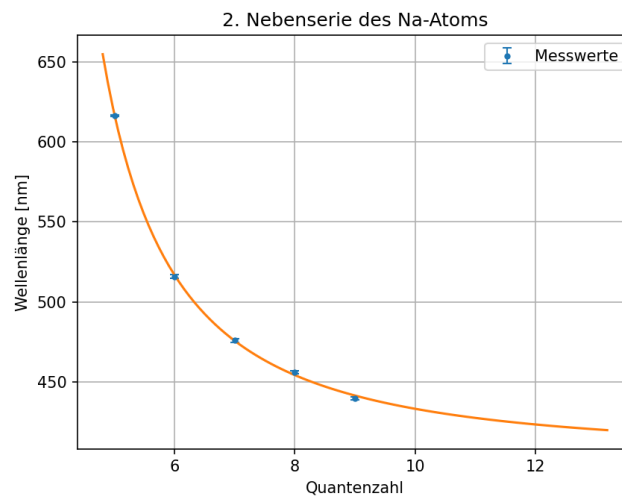
Jetzt bestimmen wir noch die χ^2 -Summe sowie die reduzierte χ^2 -Summe, um die Güte des Fits zu beurteilen.

$$\chi^2 = \sum_{i=1}^N \frac{(\lambda_{theo,i} - \lambda_{exp,i})^2}{\Delta \lambda_{exp,i}^2} \quad \chi_{red}^2 = \frac{\chi^2}{f} \quad (3.10)$$

Hierbei ist N die Anzahl der Datenpunkte und f die Anzahl der Freiheitsgrade. Außerdem können wir mit Python noch die Fitwahrscheinlichkeit berechnen. Wir erhalten:

$$\chi^2 = 2.5 \quad \chi_{red}^2 = 0.4 \quad P_{fit} = 86.0 \%$$

Die zweite Nebenserie erfolgt analog: Diesmal wird Gleichung (3.7) an die Handvoll Messergebnisse aus Tabelle 3.3 gefittet.

**Abb. 3.9:** Fitten der Messergebnisse aus Tabelle 3.3 an Funktion 3.7

$$E_{Ry} = (-15.4 \pm 2.5) \text{ eV} \quad E_{3p} = (-3.060 \pm 0.032) \text{ eV}$$

$$\Delta_s = 1.16 \pm 0.25$$

Tabelle 3.5: Vergleich der Fitergebnisse mit Literaturwerten/berechneten Größen

Größe	Vergleichswert	Fit-Wert	σ_{Abw}	proz. Abw. [%]
E_{Ry} [eV]	-13.605 ± 0	-15.4 ± 2.5	0.7	13.2
E_{3p} [eV]	-3.0256 ± 0.0028	-3.060 ± 0.032	1.1	1.13
Δ_s	1.3715 ± 0.0010	1.16 ± 0.25	0.8	16.7

$$\chi^2 = 6.3$$

$$\chi^{\text{red}} = 3.2$$

$$P_{\text{fit}} = 4.0 \%$$

4. Diskussion

Anmerkung: Die Ergebnisse wurden schon zu genüge in Abschnitt 3.(Auswertung) behandelt und hervorgehoben.

4.1. Diskussion

Sonnenspektrum

Die größte Intensität liegt im sichtbaren Bereich, sinnvoll, wenn man aus evolutionärer Perspektive darauf blickt (höhö).

Interessant ist, dass die die Alpha- und Beta-Linien der Balmerreihe im Sonnenspektrum perfekt passen, während für die Gamma und Delta-Linien zwar in deren Umgebung Emissionsliniendips zu sehen sind, diese jedoch um ein paar Nanometer verschoben sind. Grund hier für ist eventuell die Wahrscheinlichkeit von δ und γ Übergängen geringer, sodass diese Linien im Spektrum allgemein nicht so stark ausgeprägt sind.

Die Abweichungen zwischen gemessenen und theoretischen Werten der Fraunhoferlinien in Tabelle 3.1 sind sehr gering, was natürlich rein rechnerisch an dem hoch geschätzten Fehler $\Delta\lambda = 1 \text{ nm}$ liegt. Diesen hätte man auch niedriger wählen können, denn die Bestimmung der Peaks erfolgte mit Zoomen und genauem Aimen der Maus.

Eventuell liegt ein Ungenauigkeitsfaktor in der Auflösung des Gitterspektrometers. Einen systematischen Kalibrierungsfehler kann man ausschließen, da die gemessenen Peaks nicht systematisch in derselben Richtung verschoben sind. Meinem naiven Denken nach, hat Hintergrundrauschen keinen Effekt, der diese Wellenlängenabweichungen erklären könnte.

verschiedene Lichtquellen

Eine der schönsten und befriedigsten Grafiken bisher: Die Lichtquellenspektren und Erwartungen erfüllt zu sehen, ist toll. Es sind eigentlich keine großen Überraschungen, bis auf die Linie der weißen LED, bei der ich erstmal nachlesen musste, ob es eine sinnvolle Erklärung gibt. Leider haben wir nicht aufgeschrieben, ob wir das Licht eher als kalt oder warm empfunden haben. Die mittlere Wellenlänge liegt nämlich überraschend weit im gelblichen Bereich, dafür dass das Spektrum wenig Rot-Intensität aufweist.

Natriumdampflampe

In Tabelle 3.3 ist zu erkennen, dass die Abweichungen der 1.Nebenserie ziemlich gering sind, während sie bei der 2.Nebenserie deutlich angestiegen sind. Ein Grund ist hierfür evtl. die geringere Intensität dieser Peaks, da es statistisch seltener zu diesen Übergängen kommt (2.Nebenserie ?). Bei der 2.Nebenserie wurden erstens nicht alle berechneten Emissionslinien wiedergefunden, sondern zeigen zweitens auch eine deutlich höhere, diesmal statistisch signifikante Abweichung.

Fitten

Wie man schön in Abschnitt 3.3.6 sieht, ist der beim Fitten bestimmte Korrekturfaktor $\Delta_d = 0.0031 \pm 0.0023$ ziemlich klein, weswegen er bei der Berechnung in Abschnitt 3.3.2. (Berechnung der 1. Nebenserie), Gleichung (3.4) anfangs ignoriert wurde. Ob dieser Wert wirklich vernachlässigbar klein ist, kann ich nicht beurteilen. Inwiefern der nicht perfekte Fit hier einen Effekt hat, weiß ich nicht.

Die Vergleiche der Fitergebnisse in Tabelle 3.4 und Tabelle 3.5 zeigen, dass der erste Fit ziemlich gut ist. Der zweite jedoch ziemlich schlecht, auch wenn die Sigma-Abweichungen es anders erscheinen lassen (diese werden nur durch hohe Fehler nach unten gedrückt). Die prozentualen Abweichungen bei Fit 2, z.b. der Rydbergenergie liegt bei 13.2 % und die des Korrekturfaktors Δ_d bei 16.7 %. Man sieht also den fortgesetzten Trend, den ich oben schon beschrieben habe, dass die Zuordnung und Abweichung der Linien der 2. Nebenserie deutlich ungenauer als bei der 1. Nebenserie war.

Durch die niedrige Anzahl N an Datenpunkten sind die Freiheitsgrade f ziemlich niedrig, was eine Interpretation der χ Werte erschwert.

4.2. Fazit

Der Versuch hat sehr schöne Prinzipien behandelt und schöne Ergebnisse (zumindest Diagramme) produziert. Die Spektren zu untersuchen und theoretisches Wissen bestätigt zu wissen, war ziemlich cool. Auch das Vibe-Coden der Peak-Analysis Funktion war eine ziemlich lehrreiche Erfahrung (hab davor noch nicht mit Python gearbeitet). Alle Kollegen haben mir erzählt, sie hätten einfach irgendwelche Fehler hingeschrieben, tja jetzt habe ich halt 5h an dieser Funktion gesessen, um dann in 5 min Wellenlängen und FWHM instant bestimmen zu können. Hätte ich mir die Zeit sparen können, in der ich die Peaks noch per Hand mit Schreiben des Arrays `<peaks>` mit `plt.vlines(<peaks>)` geplottet habe.

Me trying to find all emission lines



Abb. 4.1: my aiming skills und gaming-mouse finally paying off ⁵

Literaturquellen

¹Veritasium, *Why It Was Almost Impossible to Make the Blue LED*, [Online; Stand 03/2026], <https://www.youtube.com/watch?v=AF8d72mA41M&t=2s> (siehe S. 1).

²Dr. J. Wagner, *Physikalisches Praktikum 2.1 für Studierende der Physik B.Sc.* [Online; Stand 03/2026] (Universität Heidelberg, März. 2026), https://git.physi.uni-heidelberg.de/schwierz/Versuchsanleitungen/src/branch/main/PAP2_1.pdf (besucht am 11.03.2026) (siehe S. 5, 6, 8, 13).

³Dr. J. Wagner, *Physikalisches Praktikum 2.2 für Studierende der Physik B.Sc.* [Online; Stand 04/2026] (Universität Heidelberg, Apr. 2026), https://git.physi.uni-heidelberg.de/schwierz/Versuchsanleitungen/src/branch/main/PAP2_2.pdf (besucht am 14.04.2026) (siehe S. 10).

Abbildungsquellen

⁴Kung Fu Panda, *My Time Has Come*, [Online; 07/2026], https://en.meming.world/wiki/My_Time_Has_Come (siehe S. 1).

⁵H*ck No, *Sweaty Speedrunner*, [Online; Stand 03/2026], <https://knowyourmeme.com/memes/sweaty-speedrunner> (siehe S. 3, 24).

⁶Degreeen, Baba66, *Sonne Strahlungsintensitaet*, [Online; Stand 03/2026], https://commons.wikimedia.org/wiki/File:Sonne_Strahlungsintensitaet.svg?lang=de (siehe S. 6).

A. Rayleigh-Streuung

Weil diese Bilder so schön sind, hier noch ein weiteres.

B. Code-Anhang

Die .jupyter Datei, mit vollständigem Code (an manchen Stellen ist etwas in der pdf-Datei abgeschnitten), findet sich auch in der [Heibox](#). Leider wird der Klick Progress in der .html Datei nicht gespeichert, deswegen sieht man leider nicht alle Bilder mit der Halbwertsbreite, ich habe eines beispielsweise in eine Markdownzelle reinkopiert.

```
In [1]: %matplotlib widget
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
import numpy as np
import scipy.constants as c
from scipy.optimize import curve_fit
from scipy.stats import chi2
from pathlib import Path
from adjustText import adjust_text
from matplotlib.widgets import Button

def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))
```

```
In [2]: base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/"234")      # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)
```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Messwerte\234

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Auswertungen\234\Code\Diagramme

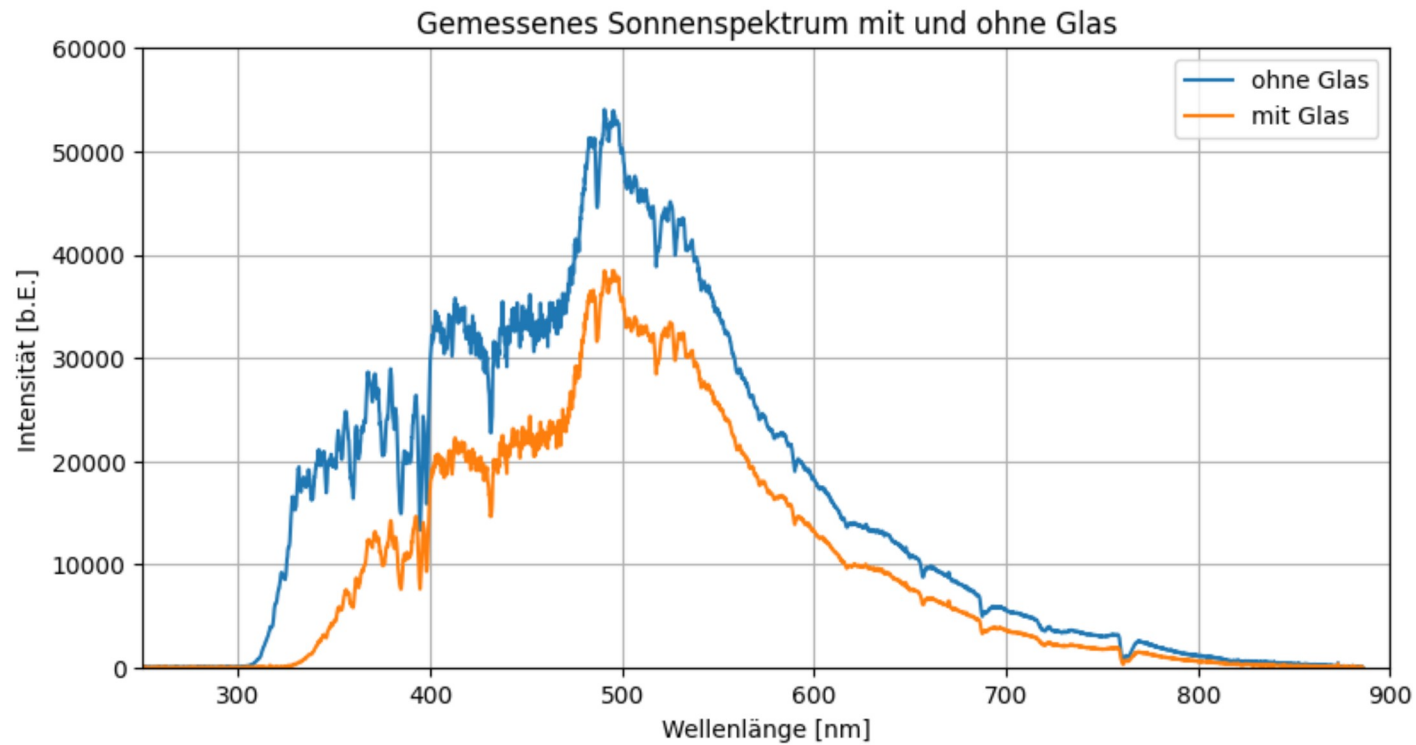
Nr.1 Sonnenspektrum mit und ohne Glasfenster

```
In [ ]: lambda_ohneGlas, intensity_ohneGlas = np.loadtxt(Messwerte_path/"Nr.1"/"blauerhimmel_ohneglas.txt", skiprows=17,
converters={0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)

lambda_mitGlas, intensity_mitGlas = np.loadtxt(Messwerte_path/"Nr.1"/"blauerhimmel_hinterglas.txt", skiprows=17,
converters={0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)

plt.figure(figsize=figsize_cm(12,6))      # Standard Plot erstellen, der gespeichert wird.
plt.plot(lambda_ohneGlas, intensity_ohneGlas, label='ohne Glas')
plt.plot(lambda_mitGlas, intensity_mitGlas, label='mit Glas')
plt.title('Gemessenes Sonnenspektrum mit und ohne Glas')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.legend()
plt.grid()
plt.ylim((0,60000))
plt.xlim((250,900))
# plt.savefig(Ausgabe_path/"Nr.1_Himmel_m_o_G.png", format="png")
```

Out[]: (250.0, 900.0)



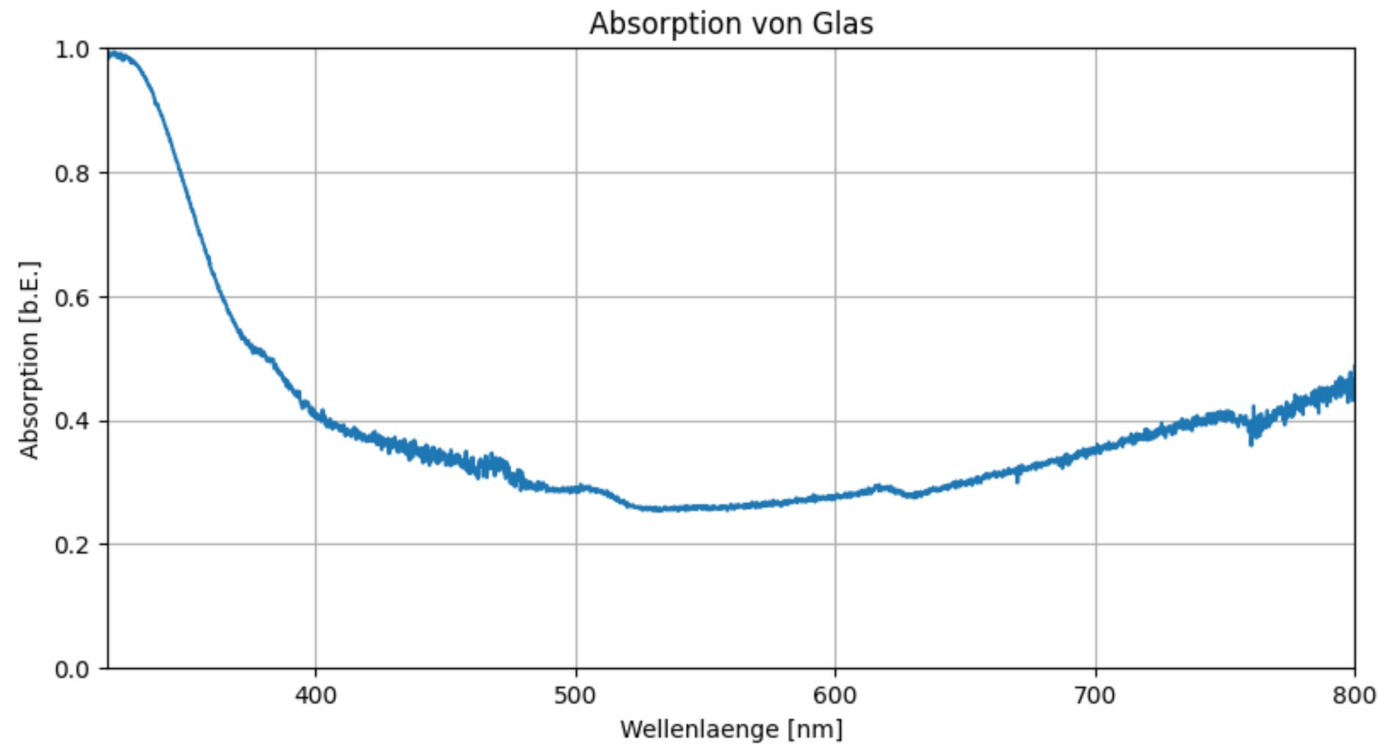
Nr.1.1 Absorption

In [4]: $A = 1 - \text{intensity_mitGlas} / \text{intensity_ohneGlas}$

```
plt.figure(figsize=figsize_cm(12,6))
plt.plot(lambda_ohneGlas,A)
plt.title('Absorption von Glas')
plt.xlabel('Wellenlaenge [nm]')
plt.ylabel('Absorption [b.E.]')
plt.grid()
plt.ylim((0,1))
plt.xlim((320,800))
# plt.savefig(Ausgabe_path/"Nr.1_Absorption_Glas.png", format="png")
```

Out[4]: (320.0, 800.0)

Figure



Nr.1.2 Hier wird es das erste Mal interessant: Sonnenspektrum mit Absorptionslinien und Balmer-Serie

```
In [ ]: lambda_sonneohneglas, intensity_sonneohneglas = np.loadtxt(Messwerte_path/"Nr.1"/"direktesonne_ohne_glas.txt", skiprows=17,
converters={0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)

bottom, top = plt.ylim()
plt.figure(figsize=figsize_cm(12, 6))
plt.plot(lambda_sonneohneglas, intensity_sonneohneglas,)
plt.vlines(x=[394, 397, 412, 431, 486.8, 517, 527, 589, 616.5, 656, 687, 760.5], ymin=0, ymax=max(bottom, top), colors='black', linewidths=0.7, linestyle='dotted', label='geschä
plt.title('Sonnenspektrum mit abgelesenen Absorptionslinien')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.ylim((0, 60000))
plt.xlim((350, 800))
balmerserie = {
    r'$H\_alpha$': 656.1,
    r'$H\_beta$': 486.1,
    r'$H\_gamma$': 434,
    r'$H\_delta$': 410.1
}
# in dieser Liste werden die Namen der einzuzeichnenden Linien gespeichert

plt.vlines(x=[410.1, 434, 486.1, 656.1], ymin=0, ymax=40000, colors='red', linewidths=0.7, label='Balmerserie')
for name, wl in balmerserie.items():
    # mit dieser Schleife werden die Linienlabels gesetzt
```



```

if not wl==410.1:
    plt.text(wl+2,15000,f"{name}\n({wl} nm)",color='red')
else:
    plt.text(wl+2,24000,f"{name}\n({wl} nm)",color='red')
texts=[]
geschätzte_absorptionslinien=[394.6,398,411.4,431.5,435.2,486.8,517.7,527.5,586.4,589.9,589.9,656.6,687.5,760.8] # mit der Maus erfasste Linien werden gespeichert und gezeichnet
for i in geschätzte_absorptionslinien:
    t = plt.text(i, 50000, str(i), color="black", rotation=0,
        verticalalignment="bottom", horizontalalignment="left")
    texts.append(t)
adjust_text(texts) # soll Labels automatisch uncolliding machen, klappt manchmal
plt.axvline(x=586.5,color='goldenrod',label='Helium (Lit.  $\lambda=487.6\text{nm}$ ', linewidth=1)
plt.text(560,40000,str(486.5), color='goldenrod')
# plt.grid()
plt.legend()
# plt.savefig(Ausgabe_path/"Nr.1_FraunhoferLinien.png", format="png")

```

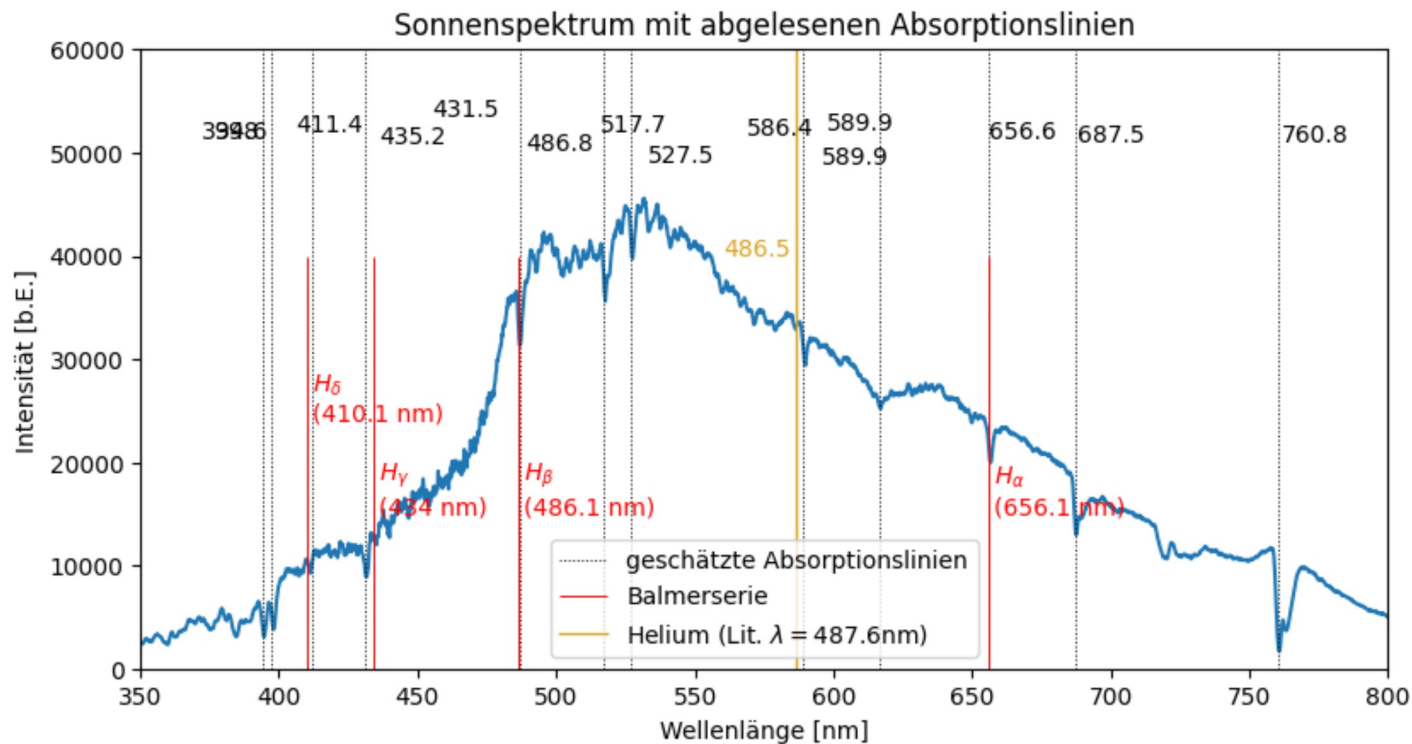
```

9 [ 0.27475631 -0.89015311]
10 [-0.47846447 -0.01190192]

```

Out[]: <matplotlib.legend.Legend at 0x1c4c450c2f0>

Figure



Nr.1.2 Sonnenspektrum mit allen Fraunhoferlinien

In [10]: plt.figure(figsize=figsize_cm(12,6))

```

plt.plot(lambda_sonneohnneglas,intensity_sonneohnneglas)
plt.title('Sonnenspektrum mit theoretischen Fraunhoferlinien')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.xlim((350,800))
plt.ylim((0,60000))
fraunhofer_lines = {
    "Sauerstoff": ("gray", [759.4, 686.7]),
    "Wasserstoff": ("red", [656.1, 486.1,434.1,410.1]),
    "Natrium": ("orange", [589.6, 589.0]),
    "Helium": ("purple", [587.6]),
    "Eisen & Calcium": ("brown", [527.0, 430.8]),
    "Magnesium": ("green", [518.4]),
    "Calcium": ("blue", [396.8, 393.4]),
}
handles=[]
for element, (color, wavelengths) in fraunhofer_lines.items():
    for wavelength in wavelengths:
        if not element=='Wasserstoff':
            plt.axvline(wavelength, color=color, linestyle="dotted",linewidth = 1)
        else:
            plt.axvline(wavelength, color=color, linewidth = 0.5)
for element, (color, _) in fraunhofer_lines.items():
    if not element=='Wasserstoff':
        handles.append(Line2D([0], [0], color=color, linestyle="dotted", label=element))
    else:
        handles.append(Line2D([0], [0], color=color, label=element))

texts=[]
plt.vlines(x=geschätzte_absorptionslinien,ymin=0,ymax=max(bottom,top), colors='black', linewidths = 0.5, label='geschätzte Absorptionslinien')
for i in geschätzte_absorptionslinien:
    t = plt.text(i, 50000, str(i), color="black", rotation=0,
        verticalalignment="bottom", horizontalalignment="left")
    texts.append(t)
adjust_text(texts)
handles.append(Line2D([0], [0], color='black', label='gemessene Linien'))
plt.legend(handles=handles,loc='center right')
# plt.savefig(Ausgabe_path/"Nr.1_FraunhoferLinientheoretisch.png", format="png")

```

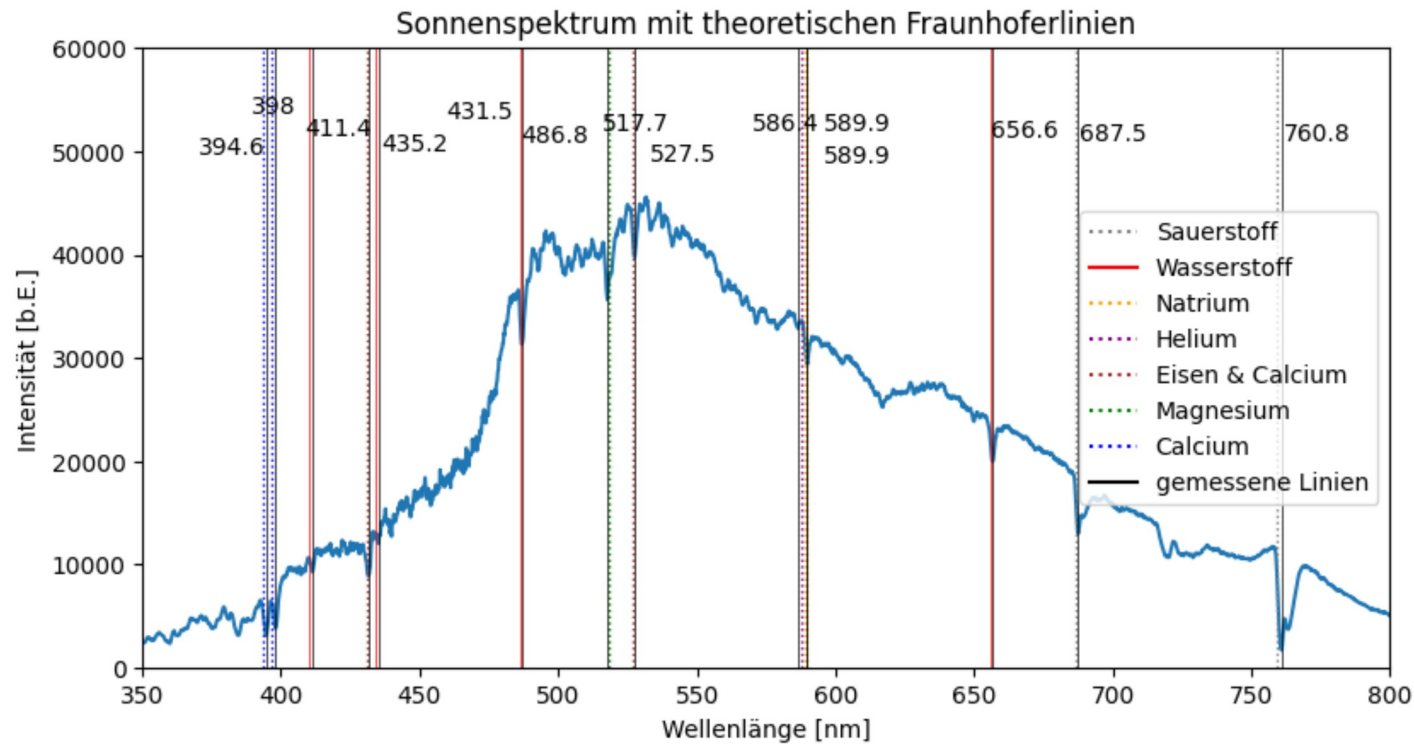
```

9 [-0.12713059  0.1726091 ]
10 [-0.17357367 -0.64139173]

```

Out[10]: <matplotlib.legend.Legend at 0x1c4c465fe00>

Figure



Nr.1.2 Ausgeben der LaTeX-Tabelle für gemessene und theoretische Fraunhoferlinien im Sonnenspektrum, berechnen der Sigma-Abweichung

```
In [56]: theoretische_Linien=[]
for element, (color, wavelengths) in fraunhofer_lines.items():
    for wavelength in wavelengths:
        theoretische_Linien.append(wavelength)
Theoretische_Linien=np.array(theoretische_Linien)
Theoretische_Linien.sort()
Geschätzte_absorptionslinien=np.array(geschätzte_absorptionslinien)
print(Theoretische_Linien)
print(Geschätzte_absorptionslinien)
sigma=np.round(abs(Theoretische_Linien-Geschätzte_absorptionslinien),1)
print(sigma)

for i,j,k in zip(Theoretische_Linien, Geschätzte_absorptionslinien, sigma):
    print(f"\num{{{i}}}&\num{{{j}}}&\num{{{k}}}\\"")
```

```
[393.4 396.8 410.1 430.8 434.1 486.1 518.4 527. 587.6 589. 589.6 656.1
686.7 759.4]
[394.6 398. 411.4 431.5 435.2 486.8 517.7 527.5 586.4 589.9 589.9 656.6
687.5 760.8]
[1.2 1.2 1.3 0.7 1.1 0.7 0.7 0.5 1.2 0.9 0.3 0.5 0.8 1.4]
\num{393.4}&\num{394.6}&\num{1.2}\\
\num{396.8}&\num{398.0}&\num{1.2}\\
\num{410.1}&\num{411.4}&\num{1.3}\\
\num{430.8}&\num{431.5}&\num{0.7}\\
\num{434.1}&\num{435.2}&\num{1.1}\\
\num{486.1}&\num{486.8}&\num{0.7}\\
\num{518.4}&\num{517.7}&\num{0.7}\\
\num{527.0}&\num{527.5}&\num{0.5}\\
\num{587.6}&\num{586.4}&\num{1.2}\\
\num{589.0}&\num{589.9}&\num{0.9}\\
\num{589.6}&\num{589.9}&\num{0.3}\\
\num{656.1}&\num{656.6}&\num{0.5}\\
\num{686.7}&\num{687.5}&\num{0.8}\\
\num{759.4}&\num{760.8}&\num{1.4}\\
```

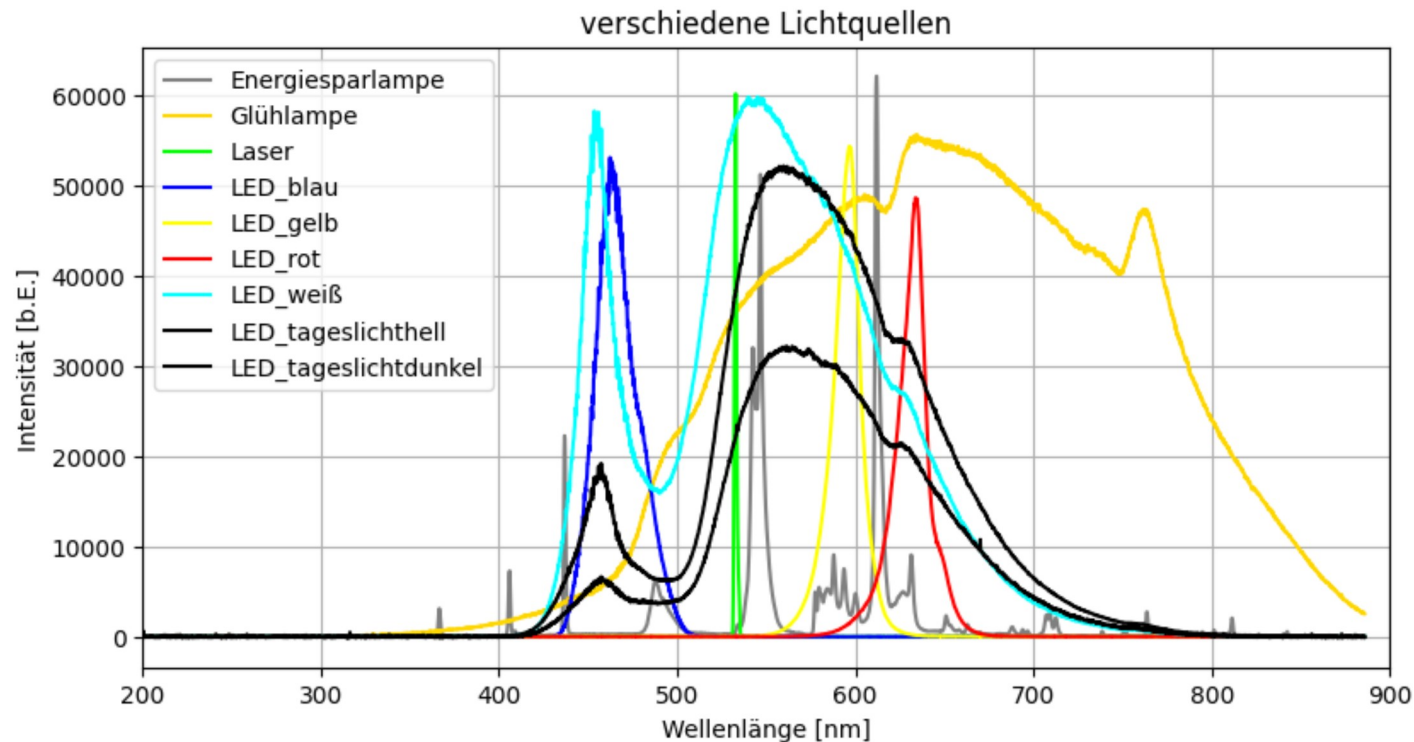
Nr.2 verschiedenen Lichtquellen

```
In [11]: # Lichtquellen-Liste mit entsprechender Farbzuteilung
lichtquellen={"Energiesparlampe":"grey", "Glühlampe":"gold", "Laser":"lime", "LED_blau":"blue", "LED_gelb":"yellow", "LED_rot":"red", "LED_weiß":"cyan", "LED_tageslichthell":

plt.figure(figsize=figsize_cm(12,6))
plt.title('verschiedene Lichtquellen')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.xlim((200,900))
for lichtquelle,color in lichtquellen.items():
    lambda_lichtquelle,intensity_lichtquelle = np.loadtxt(Messwerte_path/"Nr.2"/f"{lichtquelle}.txt",skiprows=17,
    converters= {0:comma_to_float, 1:comma_to_float},
    comments='>', unpack=True)
    plt.plot(lambda_lichtquelle, intensity_lichtquelle,label=lichtquelle,color=color)
    Lambda_lichtquelle=np.array(lambda_lichtquelle)
    Intensity_lichtquelle=np.array(intensity_lichtquelle)
    meanlambda=round(np.sum(Lambda_lichtquelle*Intensity_lichtquelle)/np.sum(Intensity_lichtquelle),2)
    print(f"{lichtquelle}&\num{{{meanlambda}}}\n\n")
plt.legend()
plt.grid()
# plt.savefig(Ausgabe_path/"Nr.2_Lichtquellen.png", format="png")
```

```
Energiesparlampe&\num{573.91}\\
Glühlampe&\num{658.13}\\
Laser&\num{534.54}\\
LED_blau&\num{467.16}\\
LED_gelb&\num{594.17}\\
LED_rot&\num{630.93}\\
LED_weiß&\num{553.64}\\
LED_tageslichthell&\num{581.49}\\
LED_tageslichtdunkel&\num{586.92}\\
```


Figure

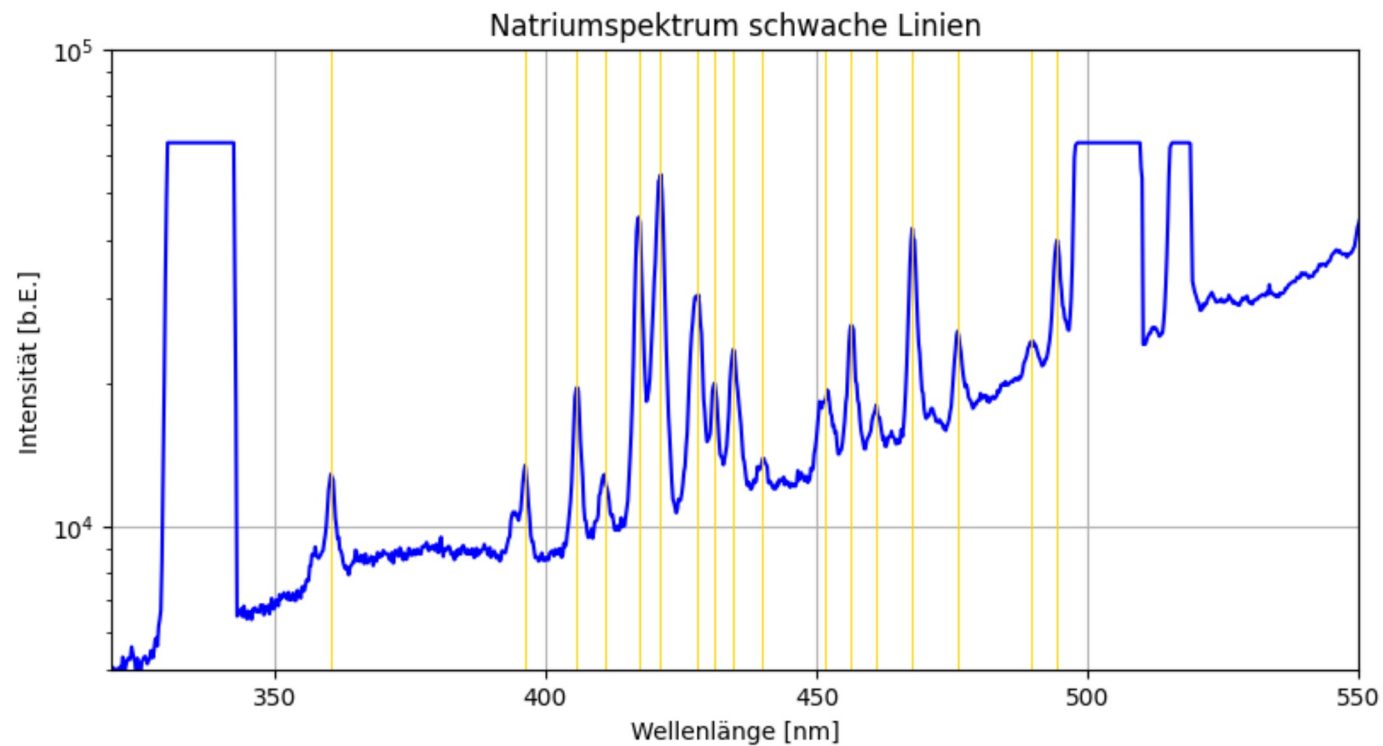


Hier geht der Abfuck los: für jedes aufgenommene Natriumspektrum wird ein Plot gezeichnet, und dann wurden per Hand die Emissionslinien gesucht, dies wurde eigentlich obsolet, da die Plots unten nochmal mit FWHM eingezeichnet wurden.

```
In [24]: lambda_natriumklein,intensity_natriumklein = np.loadtxt(Messwerte_path/"Nr.3"/"Natriumdampflampe_kleinelinie.txt",skiprows=17,
converters={0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)
color = 'blue'

natrium1klein = [360.5,396.3,405.8,411,417.2,421.2,428,431.2,434.6,440.1,451.7,456.4,461,467.6,476,489.7,494.3]
plt.figure(figsize=figsize_cm(12,6))
plt.title('Natriumspektrum schwache Linien')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.yscale('log')
plt.ylim((5000,100000))
plt.xlim((320,550))
plt.plot(lambda_natriumklein, intensity_natriumklein,color=color)
plt.vlines(x=natrium1klein,ymin=0,ymax=100000, colors='gold', linewidths = 0.7)
plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Natriumspektrumkleinelinie.png", format="png")
```

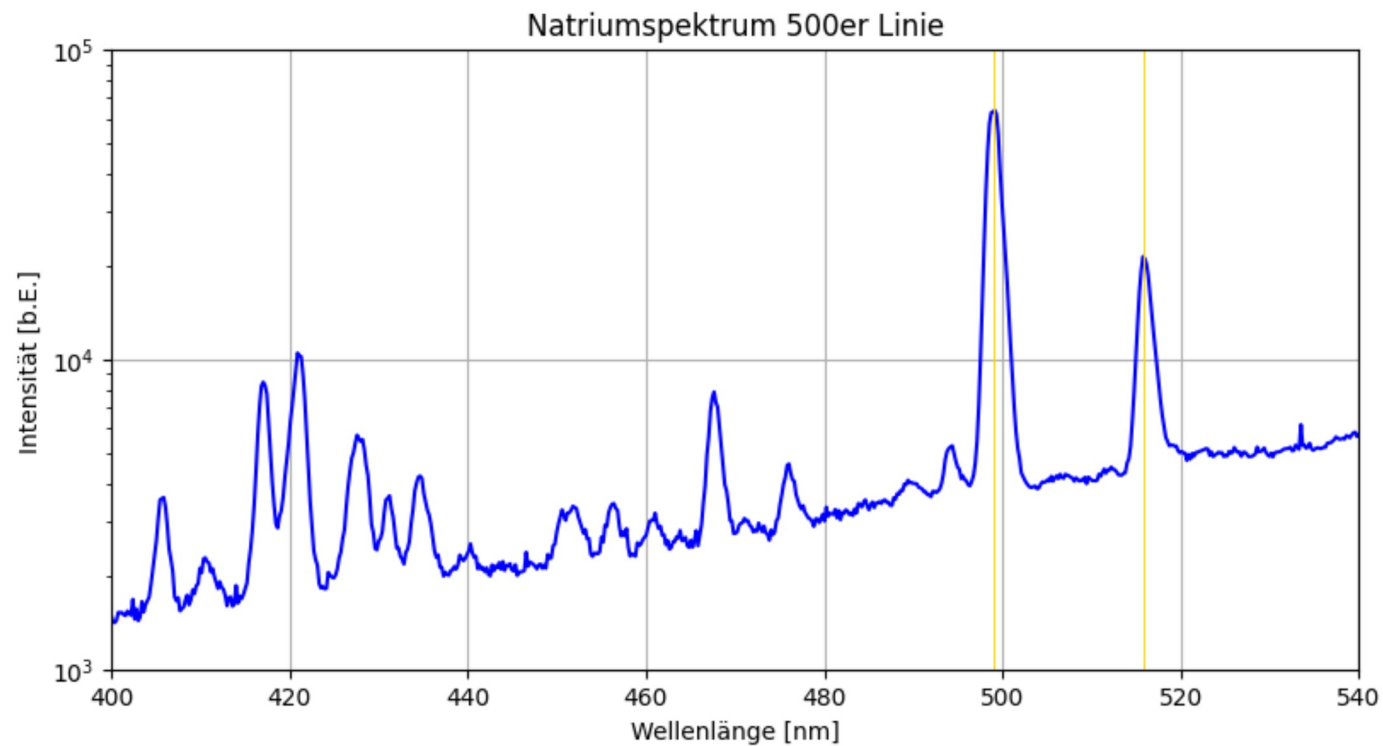
Figure



```
In [11]: lambda_natrium500,intensity_natrium500 = np.loadtxt(Messwerte_path/"Nr.3"/"Natriumdampflampe500linie.txt",skiprows=17,
converters= {0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)
```

```
Linien=[499,515.8]
plt.figure(figsize=figsize_cm(12,6))
plt.title('Natriumspektrum 500er Linie')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.yscale('log')
plt.ylim((1000,100000))
plt.xlim((400,540))
plt.plot(lambda_natrium500, intensity_natrium500,color=color)
plt.vlines(x=Linien,ymin=0,ymax=100000, colors='gold', linewidths = 0.7)
plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Natriumspektrum_500Linie.png", format="png")
```

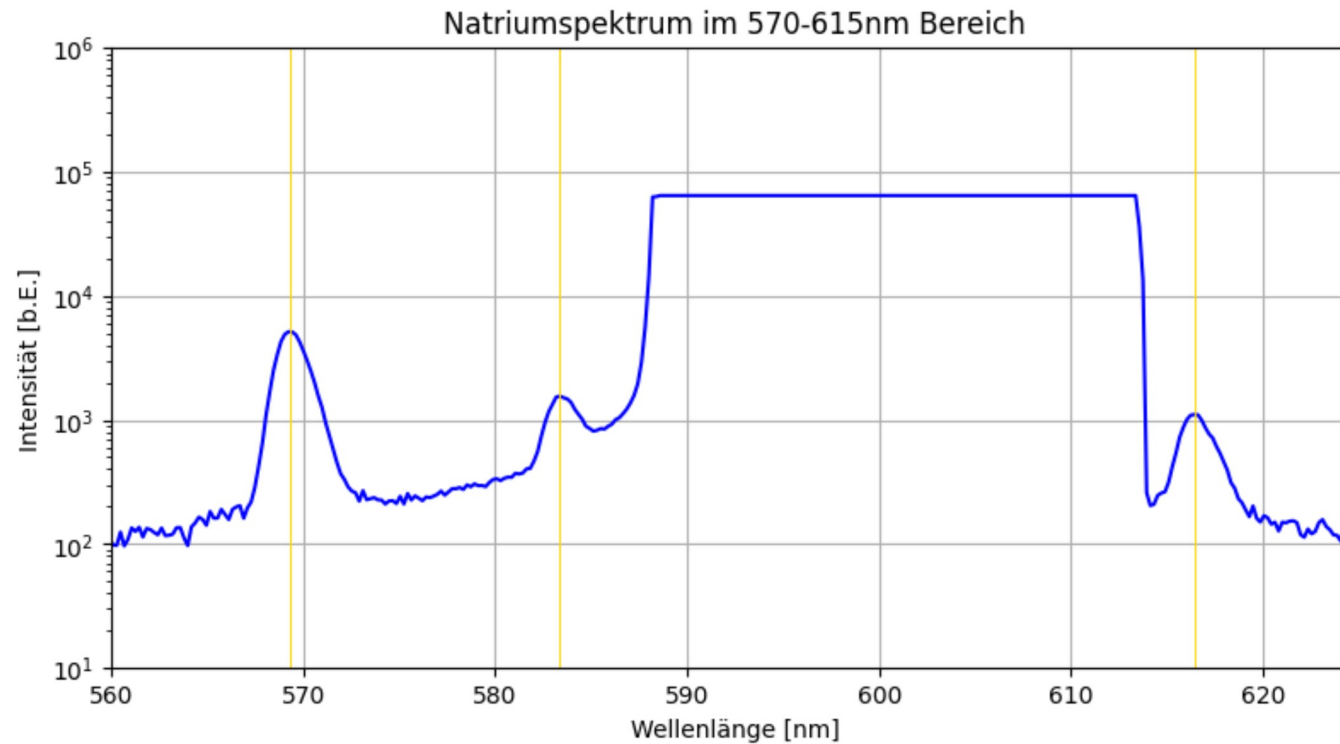
Figure



```
In [12]: lambda_natriumD,intensity_natriumD = np.loadtxt(Messwerte_path/"Nr.4"/"natrium570-615.txt",skiprows=17,
converters= {0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)
```

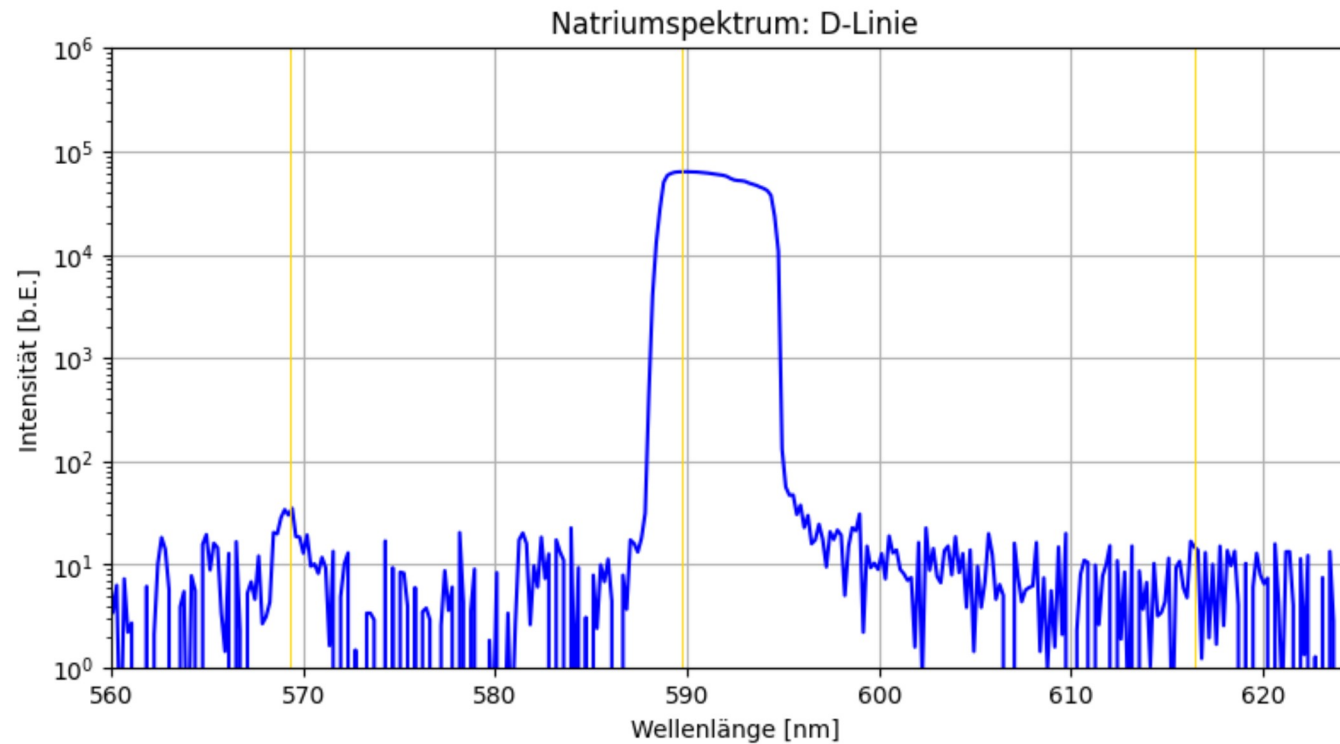
```
Linien=[569.3,583.3,616.4]
plt.figure(figsize=figsize_cm(12,6))
plt.title('Natriumspektrum im 570-615nm Bereich')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.yscale('log')
plt.ylim((10,1000000))
plt.xlim((560,625))
plt.plot(lambda_natriumD, intensity_natriumD,color=color)
plt.vlines(x=Linien,ymin=0,ymax=1000000, colors='gold', linewidths = 0.7)
plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Natriumspektrum_570-615Linien.png", format="png")
```

Figure



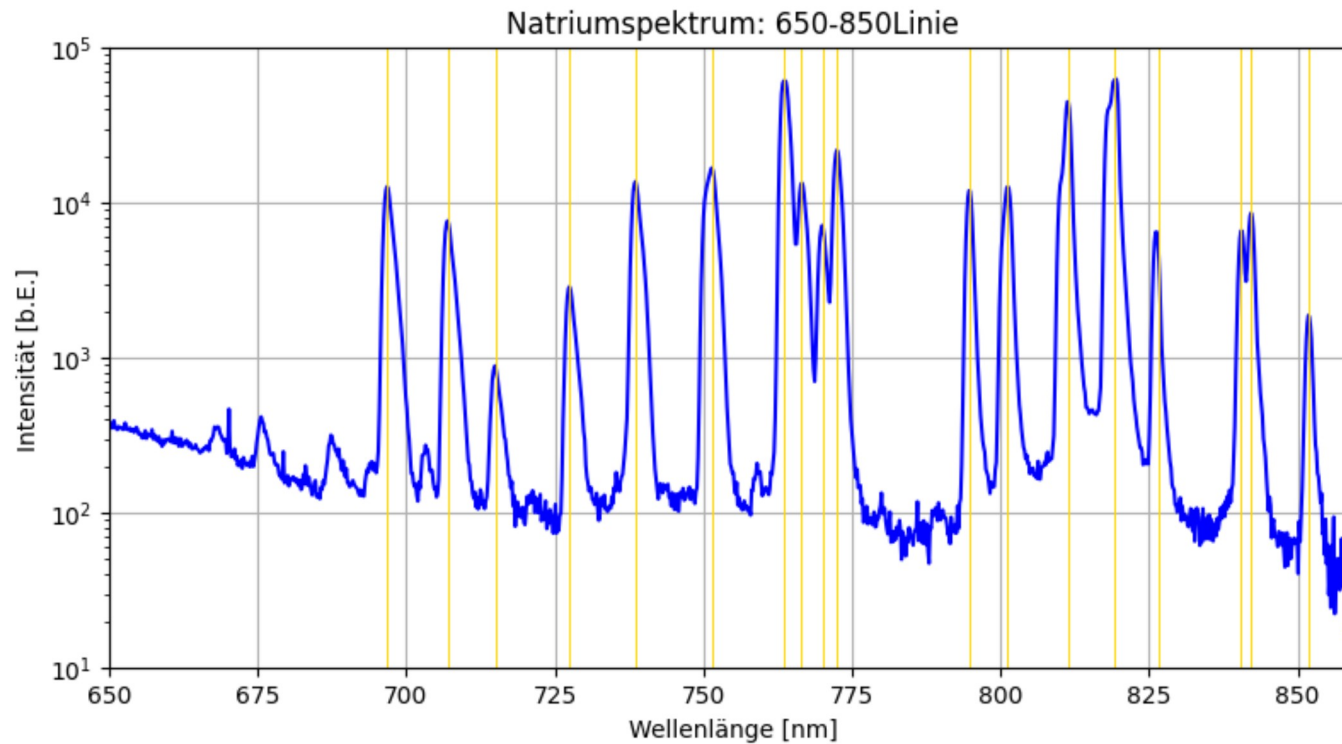
```
In [13]: lambda_natrium570_615,intensity_natrium570_615 = np.loadtxt(Messwerte_path/"Nr.4"/"NatriumD-Linie.txt",skiprows=17,
converters= {0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)
```

```
Linien=[569.3,616.4,589.7]
plt.figure(figsize=figsize_cm(12,6))
plt.title('Natriumspektrum: D-Linie')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.yscale('log')
plt.ylim((1,1000000))
plt.xlim((560,625))
plt.plot(lambda_natrium570_615, intensity_natrium570_615,color=color)
plt.vlines(x=Linien,ymin=0,ymax=1000000, colors='gold', linewidths = 0.7)
plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Natriumspektrum_D-Linie.png", format="png")
```

```
In [14]: lambda_natrium650_850,intensity_natrium650_850 = np.loadtxt(Messwerte_path/"Nr.5"/"Natrium650-850.txt",skiprows=17,
converters= {0:comma_to_float, 1:comma_to_float},
comments='>', unpack=True)

Linien=[696.7,706.9,715,727.5,738.5,751.4,763.6,766.5,770,772.5,794.7,801.2,811.3,819.3,826.6,840.5,842.2,851.8]
plt.figure(figsize=figsize_cm(12,6))
plt.title('Natriumspektrum: 650-850Linie')
plt.xlabel('Wellenlänge [nm]')
plt.ylabel('Intensität [b.E.]')
plt.yscale('log')
plt.ylim((10,100000))
plt.xlim((650,860))
plt.plot(lambda_natrium650_850, intensity_natrium650_850,color=color)
plt.vlines(x=Linien,ymin=0,ymax=100000, colors='gold', linewidths = 0.7)
plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Natriumspektrum_650-850.png", format="png")
```



Ab hier kein Bock mehr, jetzt sollten auch noch FWHM als Fehler gesucht werden smh fuck me

Das war der Anfang vom Kochen mit ChatGPT:

- funktioniert nur für eine Datei.
- Linien sind vorgegeben
- FWHM wird berechnet durch klicken
- speichert nach Stop im Ordner der jpynb-Datei als peak_results.txt. Also keine Differenzierung nach Graph, sondern nur ein einziger fester Titel

```
In [4]: # -----
# Messdaten laden
# -----
lambda_natrium570_615, intensity_natrium570_615 = np.loadtxt(
    Messwerte_path/"Nr.4"/"NatriumD-Linie.txt",
    skiprows=17,
    converters={0:comma_to_float, 1:comma_to_float},
    comments='>',
    unpack=True
)

Linien = [569.3, 616.4, 589.7]
```

```

# -----
# Figure und Plot
# -----
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
plt.subplots_adjust(bottom=0.2) # Platz für Button

ax.set_title('Natriumspektrum: D-Linie')
ax.set_xlabel('Wellenlänge [nm]')
ax.set_ylabel('Intensität [b.E.]')
ax.set_yscale('log')
ax.set_ylim((1,1000000))
ax.set_xlim((560,625))

ax.plot(lambda_natrium570_615, intensity_natrium570_615, color='blue')
ax.vlines(x=Linien, ymin=1, ymax=1000000, colors='gold', linewidths=0.7)
ax.grid()

# -----
# Variablen
# -----
clicks = []
results = []

# -----
# Datei speichern
# -----
def save_results_to_file(filename="peaks_results.txt"):
    with open(filename, "w") as f:
        for res in results:
            # Rundung auf eine Nachkommastelle
            peak = round(res['λ_peak']/2, 1)
            fwhm = round(res['FWHM'], 1)
            line = f"\\num{{{peak}}}{fwhm}} \\\\n"
            f.write(line)

# -----
# STOP-Button
# -----
def stop(event):
    # Button rot färben
    btn_stop.color = 'red'
    btn_stop.hovercolor = 'red'
    btn_stop.label.set_color('white')
    fig.canvas.draw_idle()

    save_results_to_file()
    # plt.close(fig) # optional Plot schließen

ax_stop = plt.axes([0.8, 0.01, 0.1, 0.05])
btn_stop = Button(ax_stop, 'STOP', color='lightgray', hovercolor='gray')
btn_stop.on_clicked(stop)

# -----
# Klick-Event für Peaks

```

```

# -----
def onclick(event):
    if event.inaxes != ax or event.xdata is None:
        return

    clicks.append((event.xdata, event.ydata))

    if len(clicks) == 2:
        lambda_peak = clicks[0][0]
        I2 = clicks[0][1]
        I1=0
        I_half = (I2 - I1)/2

        # Peak-Index im Array
        peak_index = np.argmin(np.abs(lambda_natrium570_615 - lambda_peak))

        # Links vom Peak den Halbwertpunkt finden
        i = peak_index
        while i > 0 and intensity_natrium570_615[i] > I_half:
            i -= 1
        lambda1 = lambda_natrium570_615[i]

        # rechts vom Peak den Halbwertpunkt finden
        i = peak_index
        while i < len(intensity_natrium570_615)-1 and intensity_natrium570_615[i] > I_half:
            i += 1
        lambda2 = lambda_natrium570_615[i]

        fwhm = lambda2 - lambda1

        # Ergebnisse speichern
        results.append({
            'λ_peak': lambda_natrium570_615[peak_index],
            'λ1': lambda1,
            'λ2': lambda2,
            'FWHM': fwhm,
            'I_half': I_half
        })

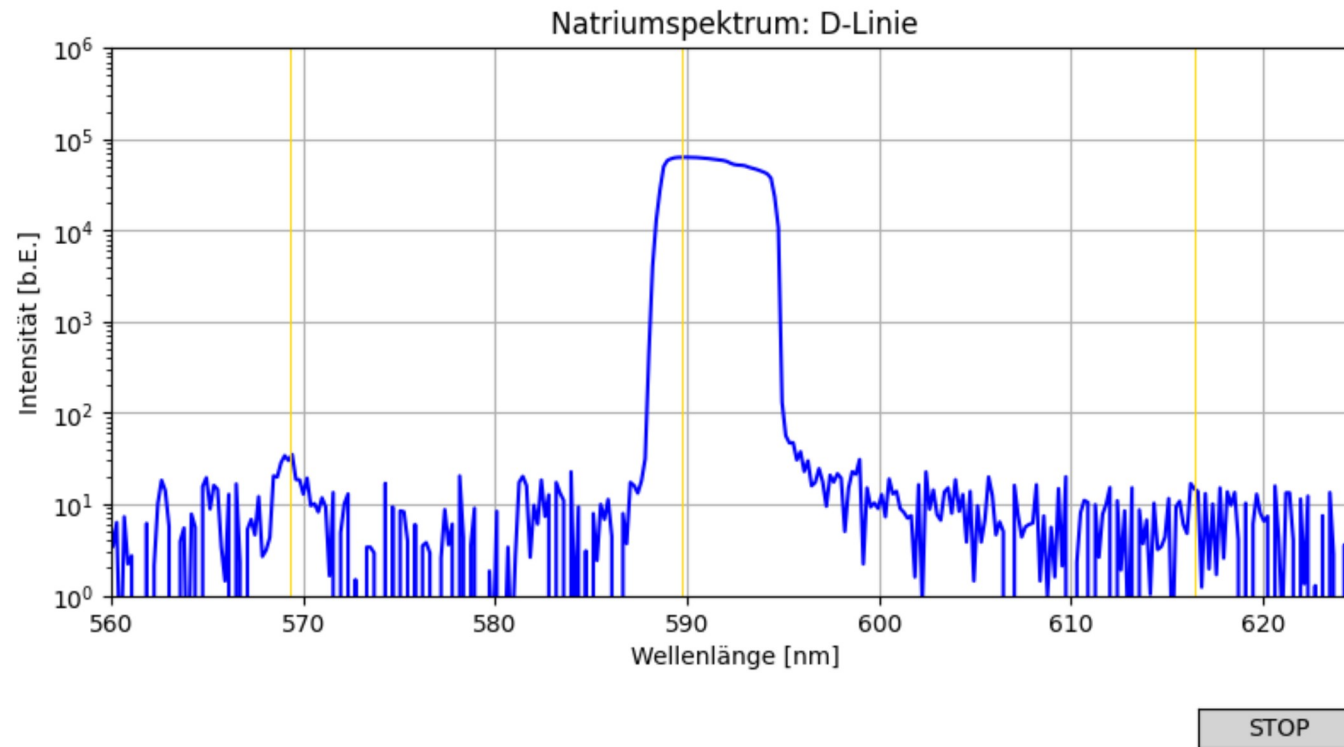
        # Linien zeichnen
        ax.hlines(I_half, xmin=lambda1, xmax=lambda2, color="red", linestyle="--")
        ax.vlines(x=lambda1, ymin=I1, ymax=I2, color="green")
        ax.vlines(x=lambda2, ymin=I1, ymax=I2, color="green")
        fig.canvas.draw_idle()

    clicks.clear() # bereit für nächsten Peak

fig.canvas.mpl_connect("button_press_event", onclick)
plt.show()

```

Figure



Das ist die finale Version für meine Anwendung. Es wird eine globale Funktion `peak_analysis` definiert.

- in graphs werden die Einstellungen zum Laden der 5 Diagramme vorgegeben
- nur FWHM wird eingezeichnet
- Werte werden gespeichert, unterm `TeX_results` Ordner als .txt Datei, je nachdem welche Messreihe man gerade analysiert.

```
In [ ]: # -----
# Messdaten laden
# -----

graphs = [
    {
        "datei": Messwerte_path/"Nr.3"/"Natriumdampflampe_kleinelinie.txt",      # Dateipfad der Messwerte des Gitterspektrometers
        "Messreihe": "kleineLinien",                                           # Name der Messreihe (ist nicht derselbe Name wie der der Messwerte Datei), wird sp
        "xlim": (350, 550),                                                    # Wellenlängebereich der Messreihe
        "ylim": (5000, 100000),                                                # Intensitätsbereich
        "Linien": [360.5,396.3,405.8,411,417.2,421.2,428,431.2,434.6,440.1,451.7,456.4,461,467.6,476,489.7,494.3] # Werte der in den Python Codes von ganz am Anf
    },
    {
        "datei": Messwerte_path/"Nr.3"/"Natriumdampflampe500linie.txt",
        "Messreihe": "500Linien",
        "xlim": (400, 540),
    }
]
```

```

        "ylim": (1000,100000),
        "Linien": [499,515.8]
    },
    {
        "datei": Messwerte_path/"Nr.4"/"natrium570-615.txt",
        "Messreihe": "570-615Linien",
        "xlim": (560,625),
        "ylim": (10,1000000),
        "Linien": [569.3,583.3,616.4]
    },
    {
        "datei": Messwerte_path/"Nr.4"/"NatriumD-Linie.txt",
        "Messreihe": "DLinien",
        "xlim": (560,625),
        "ylim": (1,1000000),
        "Linien": [569.3,616.4,589.7]
    },
    {
        "datei": Messwerte_path/"Nr.5"/"Natrium650-850.txt",
        "Messreihe": "650-850Linien",
        "xlim": (650, 860),
        "ylim": (10, 100000),
        "Linien": [696.7,706.9,715,727.5,738.5,751.4,763.6,766.5,770,772.5,794.7,801.2,811.3,819.3,826.6,840.5,842.2,851.8]
    },
]

def peak_analysis(datei_path, Messreihe, Linien, xlim=(0,1000), ylim=(0,1e6)):
    lambda_values, intensity_values = np.loadtxt(
        datei_path,
        skiprows=17,
        converters={0:comma_to_float, 1:comma_to_float},
        comments='>',
        unpack=True
    )

    results = []

    # ----- Plot -----
    fig, ax = plt.subplots(figsize=(12,6))
    plt.subplots_adjust(bottom=0.2)
    ax.plot(lambda_values, intensity_values, color='blue')
    ax.set_xlim(xlim)
    ax.set_ylim(ylim)
    ax.set_yscale('log')
    ax.set_xlabel("Wellenlänge [nm]")
    ax.set_ylabel("Intensität [b.E.]")
    ax.set_title(f"Natriumspektrum: {Messreihe}")
    ax.vlines(x=Linien, ymin=ylim[0], ymax=ylim[1], colors='gold', linewidths=0.7)
    ax.grid()

    clicks = []

    ax_btn_stop = plt.axes([0.7, 0.01, 0.2, 0.05])

# die Argumente entsprechen den Einträgen der graphs-items (Einstellungen). Übergeb
# Laden der Spalten aus der Gitterdatei in zwei Arrays, die dann zum Plotten der I(
# erstellen der results-List, in der die Ergebnisse für TeX_results gespeichert wer
# erstellen der figure (fig) und der Achsen (ax)
# I(\lambda) (zwei Spalten aus der Gitterdatei) werden geplottet
# laden des ylim-Arguments von peak_analysis
# ""
# Plottitel anhand des Messreihen arguments von peak_analysis wird festgelegt
# Emissionslinien werden geplottet, vom 0.Argument von ylim bis zum 1. Argument von
# erstellen der clicks Liste, in der die FWHM Klicks gespeichert werden
# Positionieren des Buttons relativ zu den Plot-Achsen

```

```

btn_stop = Button(ax_btn_stop, 'Press to stop measuring', color='lightgray', hovercolor='gray') # Erstellen des Buttons btn_stop

# ----- Klick-Event ----- # Klicken zur Berechnung/Einzeichnung der FWHM
def onclick(event):
    if event.button != 1: # nur Linksklick
        return
    if event.inaxes != ax or event.xdata is None:
        return
    # Toolbar-Interaktion ignorieren (Zoom oder Pan)
    if plt.get_current_fig_manager().toolbar.mode != '':
        return

    clicks.append((event.xdata, event.ydata))
    if len(clicks) == 2:
        lambda_peak = clicks[1][0]
        I1, I2 = clicks[0][1], clicks[1][1]
        I_half = I1 + (I2 - I1)/2
        peak_index = np.argmax(np.abs(lambda_values - lambda_peak))

        # suchen nach der linken Wellenlänge der FWHM
        i = peak_index
        while i > 0 and intensity_values[i] > I_half:
            i -= 1
        lambda1 = lambda_values[i]

        # suchen nach der rechten Wellenlänge der FWHM
        i = peak_index
        while i < len(intensity_values)-1 and intensity_values[i] > I_half:
            i += 1
        lambda2 = lambda_values[i]

        fwhm = lambda2 - lambda1

        results.append({
            'λ_peak': lambda_values[peak_index],
            'λ1': lambda1,
            'λ2': lambda2,
            'FWHM': fwhm,
            'I_half': I_half
        })

        ax.hlines(I_half, xmin=lambda1, xmax=lambda2, color="red")
        ax.vlines(x=lambda1, ymin=I1, ymax=I_half, color="green")
        ax.vlines(x=lambda2, ymin=I1, ymax=I_half, color="green")
        fig.canvas.draw_idle()

        clicks.clear()

# ----- Speichern -----
def save_results_to_file(filename=f"Natrium_{Messreihe}_TeXresults.txt"):
    filepath = base/"TeX_results"/ filename
    with open(filepath, "w") as f:
        for res in results:
            peak = round(res['λ_peak'], 1)

```

füllen der clicks Liste mit den Positionsdaten des Mauszeigers ('click'-event.xdata
nach zweimal drücken (also auf dem Untergrund des Peaks und dem Peak selber) wird
clicks[1][0] nimmt aus dem ersten clicks-item (xdata, ydata) das nullte Element, ...

da im Plot kontinuierliche Werte angezeigt werden, wird der nächste Datenpunkt au

für $0 < \lambda < \lambda_{peak}$ bis zu $I(\lambda) = I_{half}$ läuft die Schleife

λ_1 (links) wurde gefunden

für $\lambda > \lambda_{peak} < \max(\lambda_{ausderDatei})$ und $I(\lambda) > I_{half}$ wird gesucht

λ_2 (rechts) wurde gefunden

berechnen der FWHM

eintragen der Rechenwerte in die results Liste

Zeichnen der FWHM

keine Ahnung was das macht, irgendwie timet dies das Neuzeichnen der Grafik, wenn

es wird immer nur ein Klick gespeichert, sodass man mit harten Zahlen bei $I_1 = cli$

speichern der TeX_results Datei, anhand der Messreihen-Einstellung wird der filen
anhand des filenames wird der Speicherpfad festgelegt, relativ zu base (wurde am
results.txt wird als f geöffnet
für alle Items in der results Liste
werden die zwei in der .write benötigten Variablen benannt und mit den results Li

```

        fwhm = round(res['FWHM']/2, 1)
        f.write(f"\\num{{{peak}}}{fwhm}} \\n")

# ----- STOP-Button -----
def stop(event):
    fig.canvas.mpl_disconnect(cid)
    btn_stop.color = 'red'
    btn_stop.hovercolor = 'red'
    btn_stop.label.set_text('Measuring done')
    fig.canvas.draw_idle()
    save_results_to_file()
    # plt.close(fig) # optional
    fig.savefig(Ausgabe_path/f"Nr.3_{Messreihe}_FWHM.png", format="png")

btn_stop.on_clicked(stop)
cid = fig.canvas.mpl_connect("button_press_event", onclick)
plt.show()

```

für alle Einträge in der results Liste wird eine Zeile in die results.txt geschri

wenn man mit auswählen aller peaks fertig ist, stop drücken, danach wird alles ge

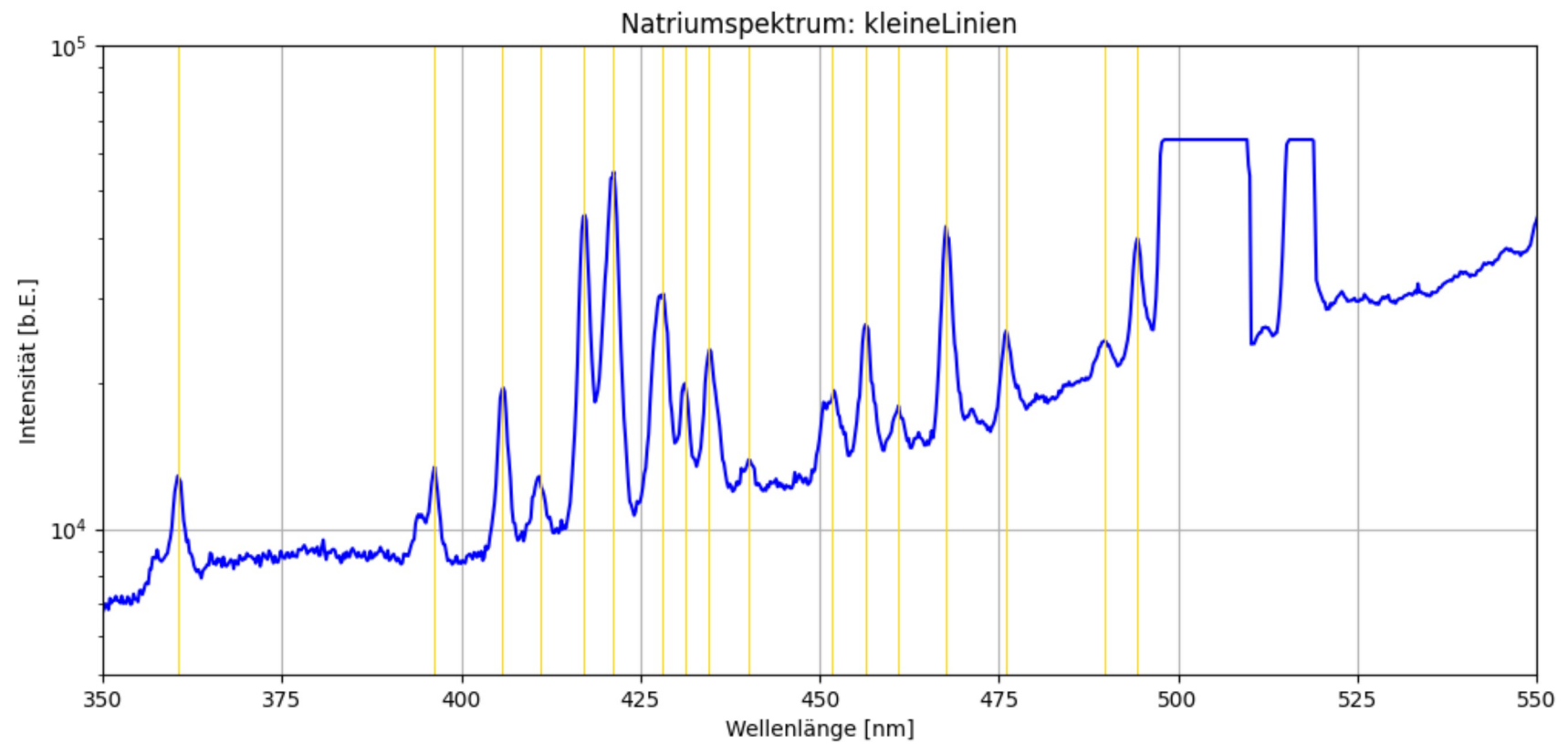
wird der Button gedrückt (also die .on_clicked-Aktion), wird die Stop Funktion au

bei jedem Klick wird neu gezeichnet

```

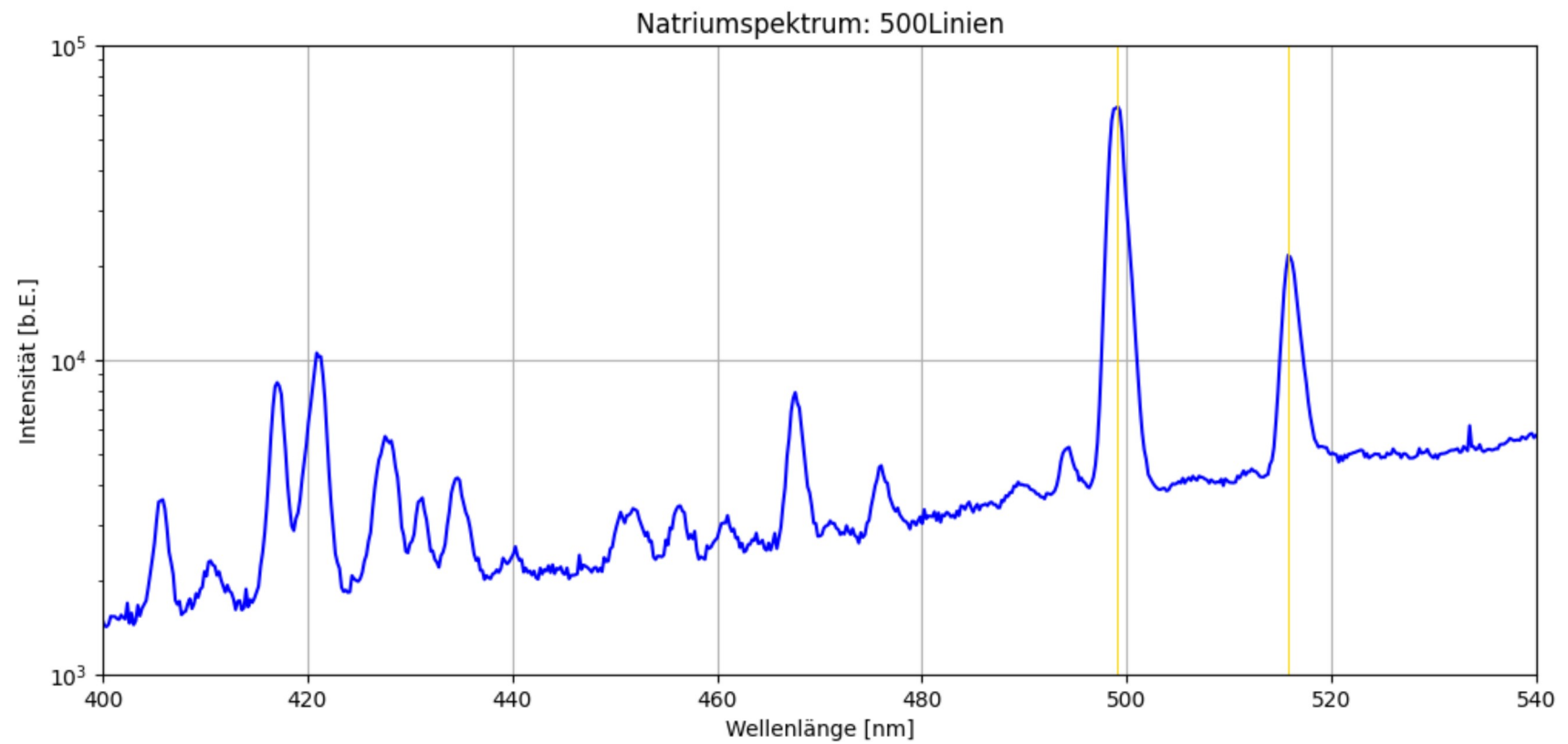
In [14]: index_graph = 0
peak_analysis(graphs[index_graph]["datei"], graphs[index_graph]["Messreihe"], graphs[index_graph]["Linien"], graphs[index_graph]["xlim"], graphs[index_graph]["ylim"])

```

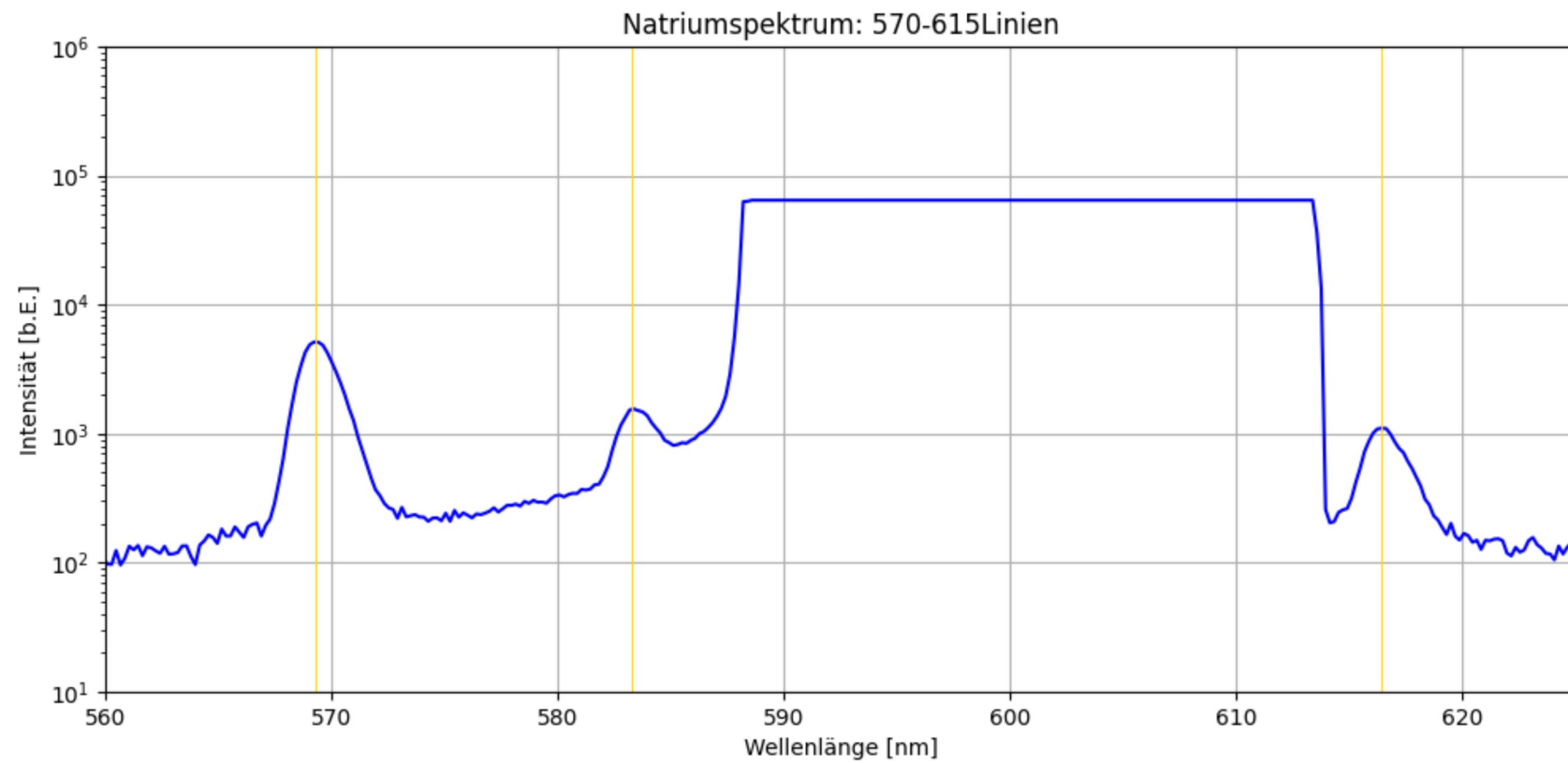
```
In [15]: index_graph = 1
peak_analysis(graphs[index_graph]["datei"], graphs[index_graph]["Messreihe"], graphs[index_graph]["Linien"], graphs[index_graph]["xlim"], graphs[index_graph]["ylim"])
```

Figure

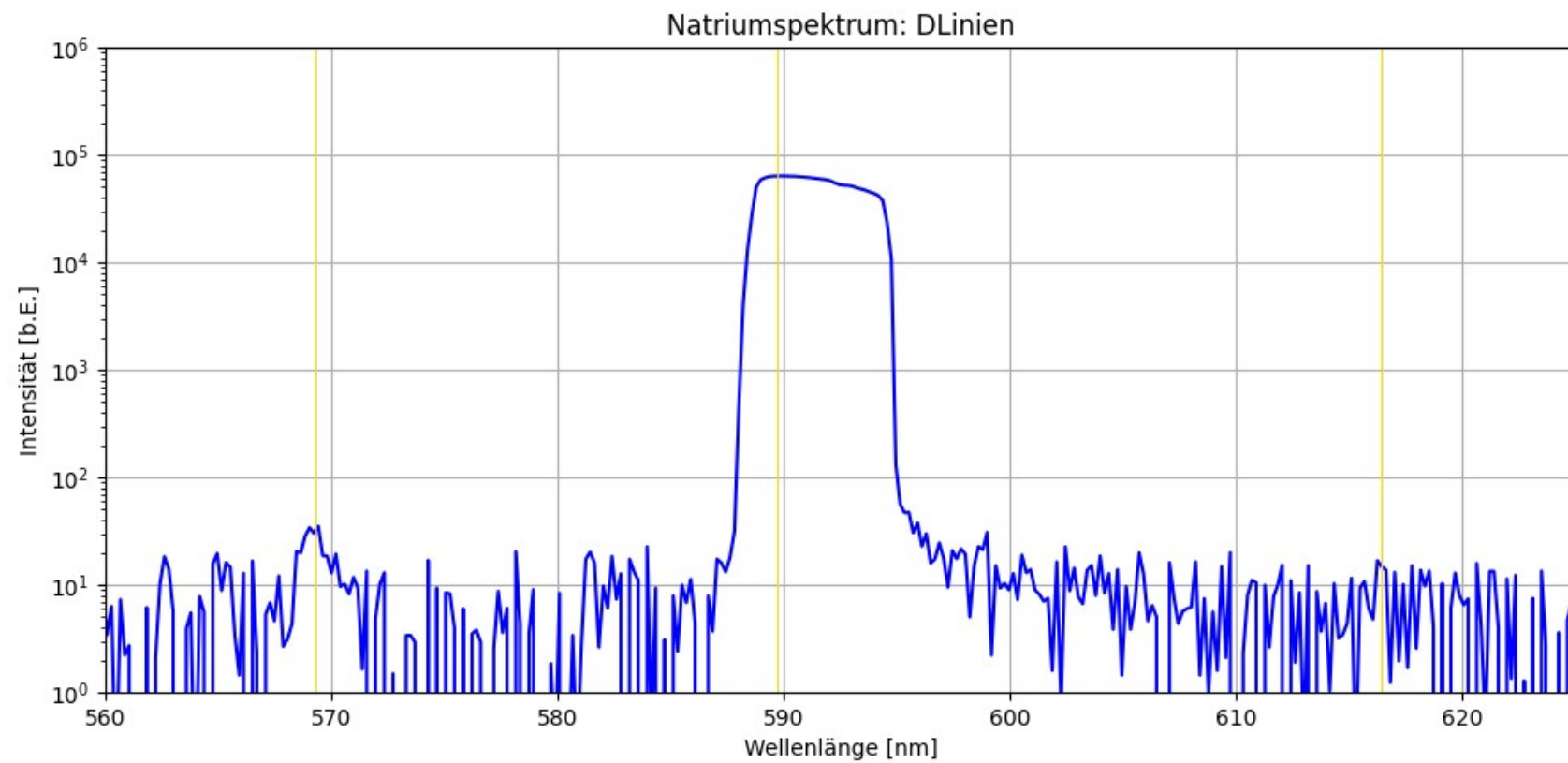


Press to stop measuring

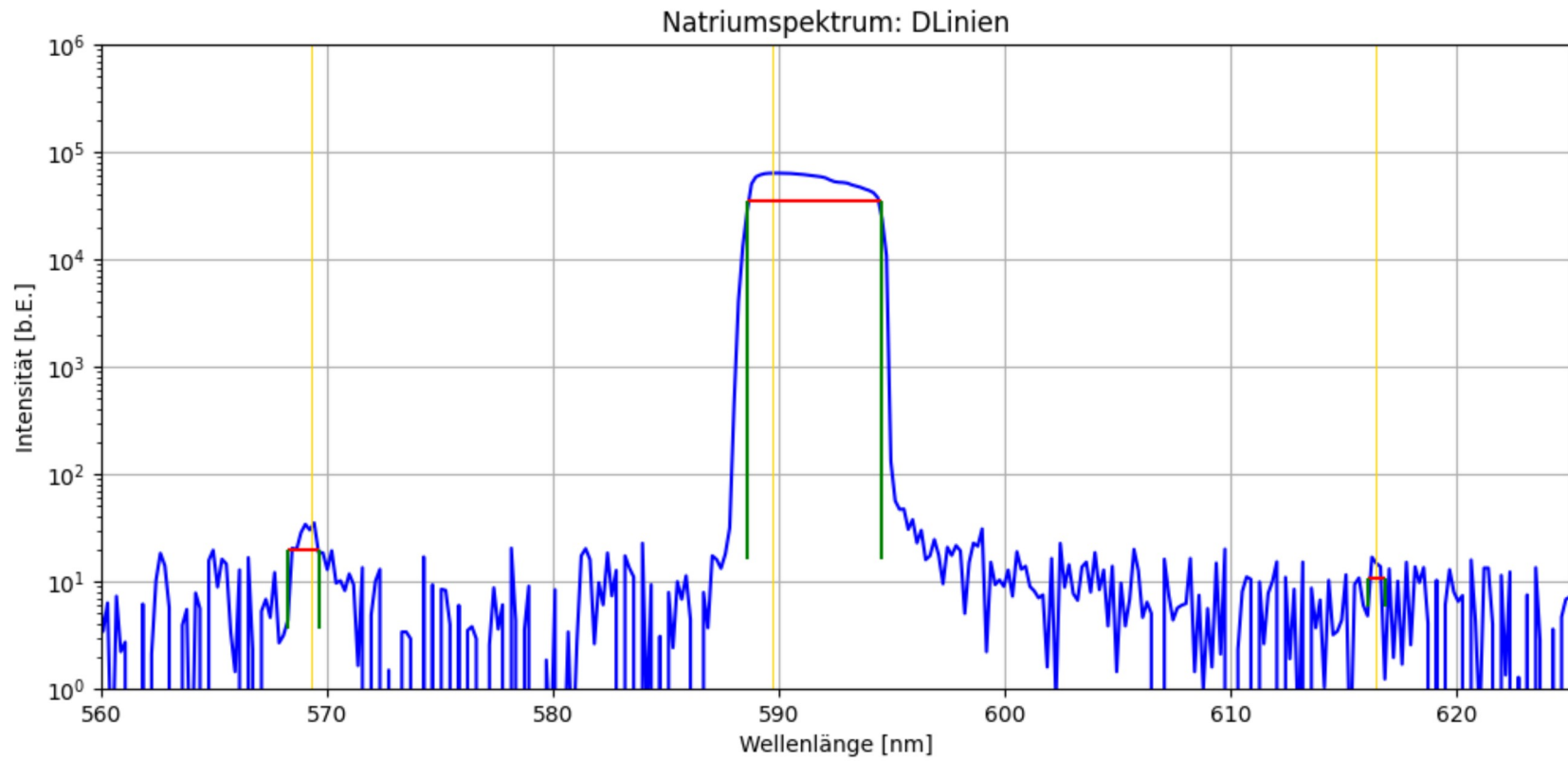
```
In [16]: index_graph = 2
peak_analysis(graphs[index_graph]["datei"], graphs[index_graph]["Messreihe"], graphs[index_graph]["Linien"], graphs[index_graph]["xlim"], graphs[index_graph]["ylim"])
```



```
In [17]: index_graph = 3
peak_analysis(graphs[index_graph]["datei"], graphs[index_graph]["Messreihe"], graphs[index_graph]["Linien"], graphs[index_graph]["xlim"], graphs[index_graph]["ylim"])
```

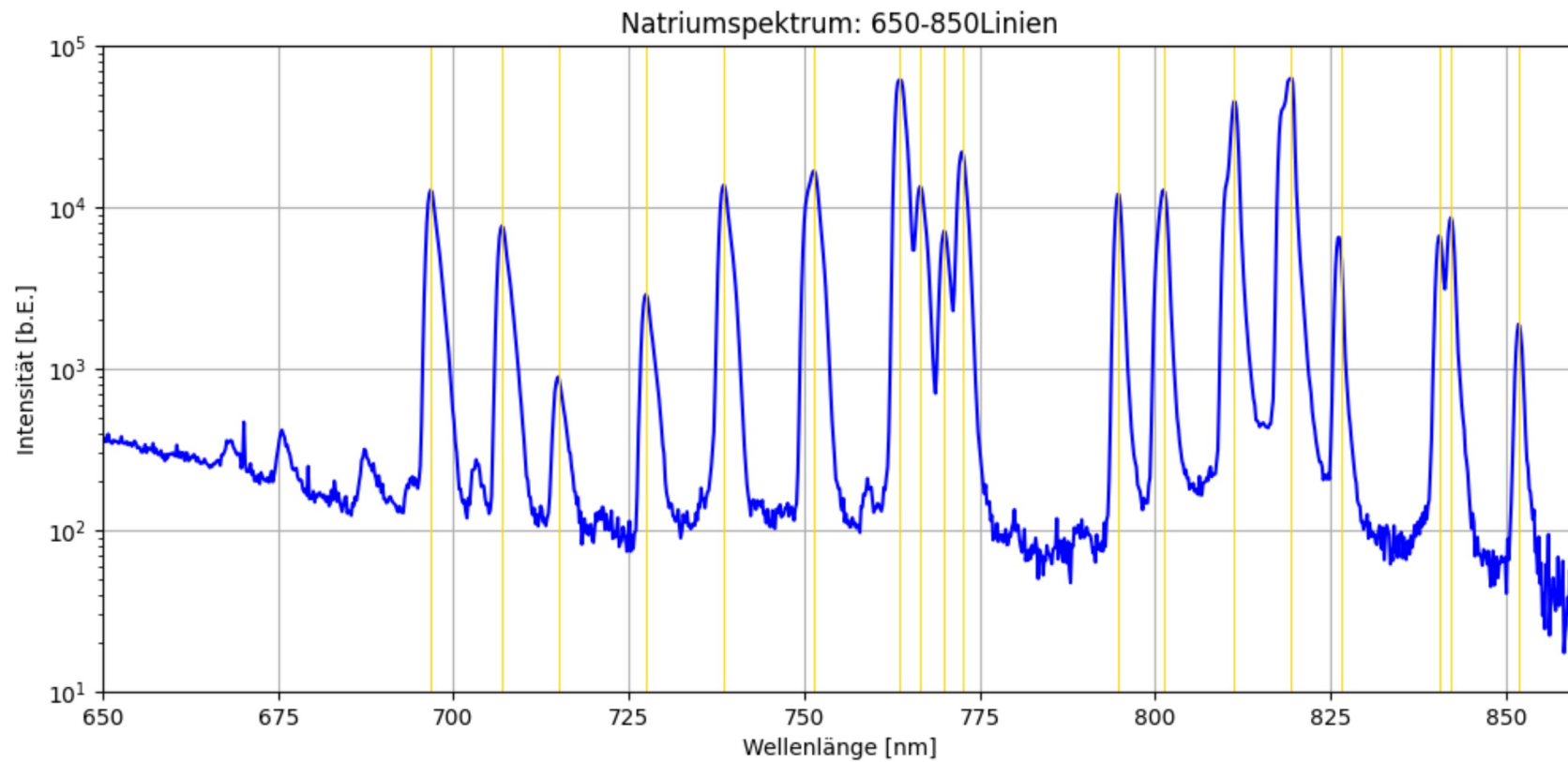


Press to stop measuring



Measuring done

```
In [18]: index_graph = 4
peak_analysis(graphs[index_graph]["datei"], graphs[index_graph]["Messreihe"], graphs[index_graph]["Linien"], graphs[index_graph]["xlim"], graphs[index_graph]["ylim"])
```



Nr.3 Berechnen der Nebenserien

Erwartete Linien für die 1. Nebenserie: md → 3p

```
In [2]: #Berechnen von E_3p
RyE = -1*c.h*c.c.c.Rydberg/c.eV
m = 3
lamb_3 = 819
Delta_lamb_3 = 1.5
E_3p = RyE/ m**2 - c.h * c.c / c.eV / (lamb_3*1e-9)
Delta_E_3p = c.h * c.c / c.eV / (lamb_3*1e-9)**2 * Delta_lamb_3*1e-9
print(f'E_3p={round(E_3p,4)}, {round(Delta_E_3p,4)}')
```

```
def lamb_m(m):
    lamb = c.h * c.c / c.eV / (RyE/ m**2 - E_3p)*1e9
    Delta_lamb = c.h * c.c * Delta_E_3p * c.eV / (RyE * c.eV / m**2 - E_3p * c.eV)**2 * 1e9
```

```

    return lamb, Delta_lamb

nebenserie1 = []
for m in range(3,14):
    l, k = lamb_m(m)
    nebenserie1.append((l,k))
    print(f'm={m:2d}, lambda = {round(nebenserie1[m-3][0],1)} ± {round(nebenserie1[m-3][1],1)}')

for m in range(3,14):
    print(f'{m}&\num{{{round(nebenserie1[m-3][0],1)}({round(nebenserie1[m-3][1],1)})}}')

```

```

E_3p=-3.0256, 0.0028
m= 3, lambda = 819.0 ± 1.5
m= 4, lambda = 570.0 ± 0.7
m= 5, lambda = 499.7 ± 0.6
m= 6, lambda = 468.3 ± 0.5
m= 7, lambda = 451.2 ± 0.5
m= 8, lambda = 440.8 ± 0.4
m= 9, lambda = 433.9 ± 0.4
m=10, lambda = 429.1 ± 0.4
m=11, lambda = 425.6 ± 0.4
m=12, lambda = 423.0 ± 0.4
m=13, lambda = 421.0 ± 0.4
3&\num{819.0(1.5)}
4&\num{570.0(0.7)}
5&\num{499.7(0.6)}
6&\num{468.3(0.5)}
7&\num{451.2(0.5)}
8&\num{440.8(0.4)}
9&\num{433.9(0.4)}
10&\num{429.1(0.4)}
11&\num{425.6(0.4)}
12&\num{423.0(0.4)}
13&\num{421.0(0.4)}

```

Erwartete Linien für die 2. Nebenserie: $m_s \rightarrow 3p$

```

In [10]: lamb_D=589
Delta_lamb_D = 1.5
E_3s = E_3p-c.c*c.h/(c.eV*lamb_D*1e-9)
Delta_E_3s = np.sqrt((Delta_E_3p)**2 + (1.2398e3/(lamb_D)**2 * (Delta_lamb_D))**2)
print(f'E_3s={round(E_3s,3)}, {round(Delta_E_3s,4)}')

korrs = 3-np.sqrt(RyE/E_3s)
Delta_korrs =np.sqrt(RyE/E_3s**3)*Delta_E_3s/2
print(f'korrs={round(korrs,4)}, {round(Delta_korrs,4)}')

def lamb_m(m):
    lamb = c.h * c.c / c.eV / (RyE/ (m-korrs)**2 - E_3p)*1e9
    Delta_lamb = np.sqrt((((2*RyE*c.c*c.eV*c.h/((-korrs + m)**3*(-E_3p*c.eV+RyE*c.eV/((-korrs + m)**2)**2))*Delta_korrs)**2 + ((c.c*c.eV*c.h/(-E_3p*c.eV+ RyE*c.eV/((-korrs + m)**2)**2))**2))**2))
    return lamb, Delta_lamb

nebenserie2 = []
for m in range(4,10):
    l, k = lamb_m(m)

```

```

nebenserie2.append((l,k))
print(f'm={m:2d}, lambda = {round(nebenserie2[m-4][0],1)} ± {round(nebenserie2[m-4][1],1)}')

for m in range(4,10):
    print(f'{m}&\num{{{round(nebenserie2[m-4][0],1)}({round(nebenserie2[m-4][1],1)})}}')

E_3s=-5.131,0.006
korrs=1.3715,0.001
m= 4, lambda = 1173.8 ± 3.5
m= 5, lambda = 622.4 ± 0.9
m= 6, lambda = 518.7 ± 0.6
m= 7, lambda = 477.6 ± 0.5
m= 8, lambda = 456.5 ± 0.5
m= 9, lambda = 444.1 ± 0.4
4&\num{1173.8(3.5)}
5&\num{622.4(0.9)}
6&\num{518.7(0.6)}
7&\num{477.6(0.5)}
8&\num{456.5(0.5)}
9&\num{444.1(0.4)}

```

Erwartete Linien für die Hauptserie: mp → 3s

```

In [11]: korrp = 3-np.sqrt(RyE/E_3p)
Delta_korrp =np.sqrt(RyE/E_3p**3)*Delta_E_3p
print(f'korrs={round(korrp,4)},{round(Delta_korrp,4)}')

def lamb_m(m):
    lamb = c.h * c.c / c.ev / (RyE / (m-korrp)**2 - E_3s)*1e9
    Delta_lamb = np.sqrt((( -2*RyE*c.c*c.ev*c.h/((-korrp + m)**3*(-E_3s*c.ev +RyE*c.ev/(-korrp + m)**2)**2))*Delta_korrp)**2 + ((c.c*c.ev*c.h/(-E_3s*c.ev+ RyE*c.ev/(-ko
    return lamb, Delta_lamb

hauptserie = []
for m in range(4,6):
    l, k = lamb_m(m)
    hauptserie.append((l,k))
    print(f'm={m:2d}, lambda = {round(hauptserie[m-4][0],1)} ± {round(hauptserie[m-4][1],1)}')

for m in range(4,6):
    print(f'{m}&\num{{{round(hauptserie[m-4][0],1)}({round(hauptserie[m-4][1],1)})}}')

korrs=0.8794,0.0019
m= 4, lambda = 332.1 ± 0.6
m= 5, lambda = 286.4 ± 0.4
4&\num{332.1(0.6)}
5&\num{286.4(0.4)}

```

Berechnung der Sigma-Abweichungen die data-Felder sind die zugeordneten Zeilen aus der LateX-Tabelle, in denen die berechneten und aus den Plots extrahierten Emissionslinien von Natrium eingetragen wurden.

```

In [ ]: import re
import math

data1 = """

```



```

3&\num{819.0(1.5)}&\num{819.1(1.1)} \\
4&\num{570.0(0.7)}&\num{569.2(1.0)} \\
5&\num{499.7(0.6)}&\num{498.9(1.0)} \\
6&\num{468.3(0.5)}&\num{467.6(0.9)} \\
7&\num{451.2(0.5)}&\num{452.2(1.7)} \\
8&\num{440.8(0.4)}&\num{439.7(1.1)} \\
9&\num{433.9(0.4)}&\num{434.7(1.0)} \\
10&\num{429.1(0.4)}&\num{428.2(1.4)} \\
13&\num{421.0(0.4)}&\num{421.1(1.1)}&\\
"""

```

```

pattern = r'\num{([\d.]+\s+)(\s+([\d.]+\s+))}'
patternm = r'^(\d+)'

```

```

nebenserie1_wellen=[]
nebenserie1_wellenfehler=[]
nebenserie1m=[]
for line in data1.strip().splitlines():
    matches = re.findall(pattern, line)
    matchm = re.search(patternm, line)
    if len(matches) == 2 and matchm:
        m = int(matchm.group(1))
        val1, err1 = map(float, matches[0])
        val2, err2 = map(float, matches[1])
        nebenserie1_wellen.append(val2)
        nebenserie1_wellenfehler.append(err2)
        nebenserie1m.append(m)
        sigma = abs(val1 - val2) / math.sqrt(err1**2 + err2**2)
        print(f"&{{sigma:.2f}}")

```

```

Nebenserie1_wellen=np.array(nebenserie1_wellen)
Nebenserie1_wellenfehler=np.array(nebenserie1_wellenfehler)
Nebenserie1m=np.array(nebenserie1m)
print(Nebenserie1m)
print(Nebenserie1_wellen)
print(Nebenserie1_wellenfehler)

```

```

print()

```

```

data2= """
5&\num{622.4(0.9)}&\num{616.2(0.4)} \\
6&\num{518.7(0.6)}&\num{515.8(1.0)} \\
7&\num{477.6(0.5)}&\num{475.8(1.1)} \\
8&\num{456.5(0.5)}&\num{456.0(1.0)} \\
9&\num{444.1(0.4)}&\num{439.7(1.1)} \\
"""

```

```

nebenserie2_wellen=[]
nebenserie2_wellenfehler=[]
nebenserie2m=[]
for line in data2.strip().splitlines():
    matches = re.findall(pattern, line)

```

```

matchm = re.search(patternm, line)
if len(matches) == 2 and matchm:
    m = int(matchm.group(1))
    val1, err1 = map(float, matches[0])
    val2, err2 = map(float, matches[1])
    nebenserie2_wellen.append(val2)
    nebenserie2_wellenfehler.append(err2)
    nebenserie2m.append(m)
    sigma = abs(val1 - val2) / math.sqrt(err1**2 + err2**2)
    print(f"&{sigma:.2f}")

```

```

Nebenserie2_wellen=np.array(nebenserie2_wellen)
Nebenserie2_wellenfehler=np.array(nebenserie2_wellenfehler)
Nebenserie2m=np.array(nebenserie2m)
print(Nebenserie2m)
print(Nebenserie2_wellen)
print(Nebenserie2_wellenfehler)

```

```

&0.05
&0.66
&0.69
&0.68
&0.56
&0.94
&0.74
&0.62
&0.09
[ 3  4  5  6  7  8  9 10 13]
[819.1 569.2 498.9 467.6 452.2 439.7 434.7 428.2 421.1]
[1.1 1.  1.  0.9 1.7 1.1 1.  1.4 1.1]

&6.30
&2.49
&1.49
&0.45
&3.76
[5  6  7  8  9]
[616.2 515.8 475.8 456.  439.7]
[0.4 1.  1.1 1.  1.1]

```

Fitten der gemessenen Ergebnisse aus der Natriumtabelle der ersten Nebenserie

```

In [29]: def fit_func1(m,E_Ry,E_3p,D_d):
          return 1.2398E3/(E_Ry/(m-D_d)**2-E_3p)
          para = [-13.6,-3,-0.02]
          popt, pcov = curve_fit(fit_func1,Nebenserie1m,Nebenserie1_wellen,sigma=Nebenserie1_wellenfehler,p0=para)
          x=np.linspace(2.8,13.2, 100)

          plt.figure(dpi=150)
          plt.errorbar(Nebenserie1m,Nebenserie1_wellen,yerr=Nebenserie1_wellenfehler,fmt=".",capsize=3,elinewidth=1,label='Messwerte')
          plt.plot(x,fit_func1(x,*popt))
          plt.xlabel('Quantenzahl')
          plt.ylabel('Wellenlänge [nm]')
          plt.title('1. Nebenserie des Na-Atoms')
          plt.legend()

```

```

plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Nebenserie1.png", format="png")

print("E_Ry=",popt[0], ", Standardfehler=", np.sqrt(pcov[0][0]))
print("E_3p=",popt[1], ", Standardfehler=", np.sqrt(pcov[1][1]))
print("D_d=",popt[2], ", Standardfehler=", np.sqrt(pcov[2][2]))

chi2=np.sum((fit_func1(Nebenserie1m,*popt)- Nebenserie1_wellen)**2/ Nebenserie1_wellenfehler**2)
dof=len(Nebenserie1m)-3 #dof:degrees of freedom, Freiheitsgrad
chi2_red=chi2/dof
print("chi2=", chi2_)
print("chi2_red=",chi2_red)

prob=round(1-chi2.cdf(chi2_,dof),2)*100
print("Wahrscheinlichkeit:", prob,"%")

```

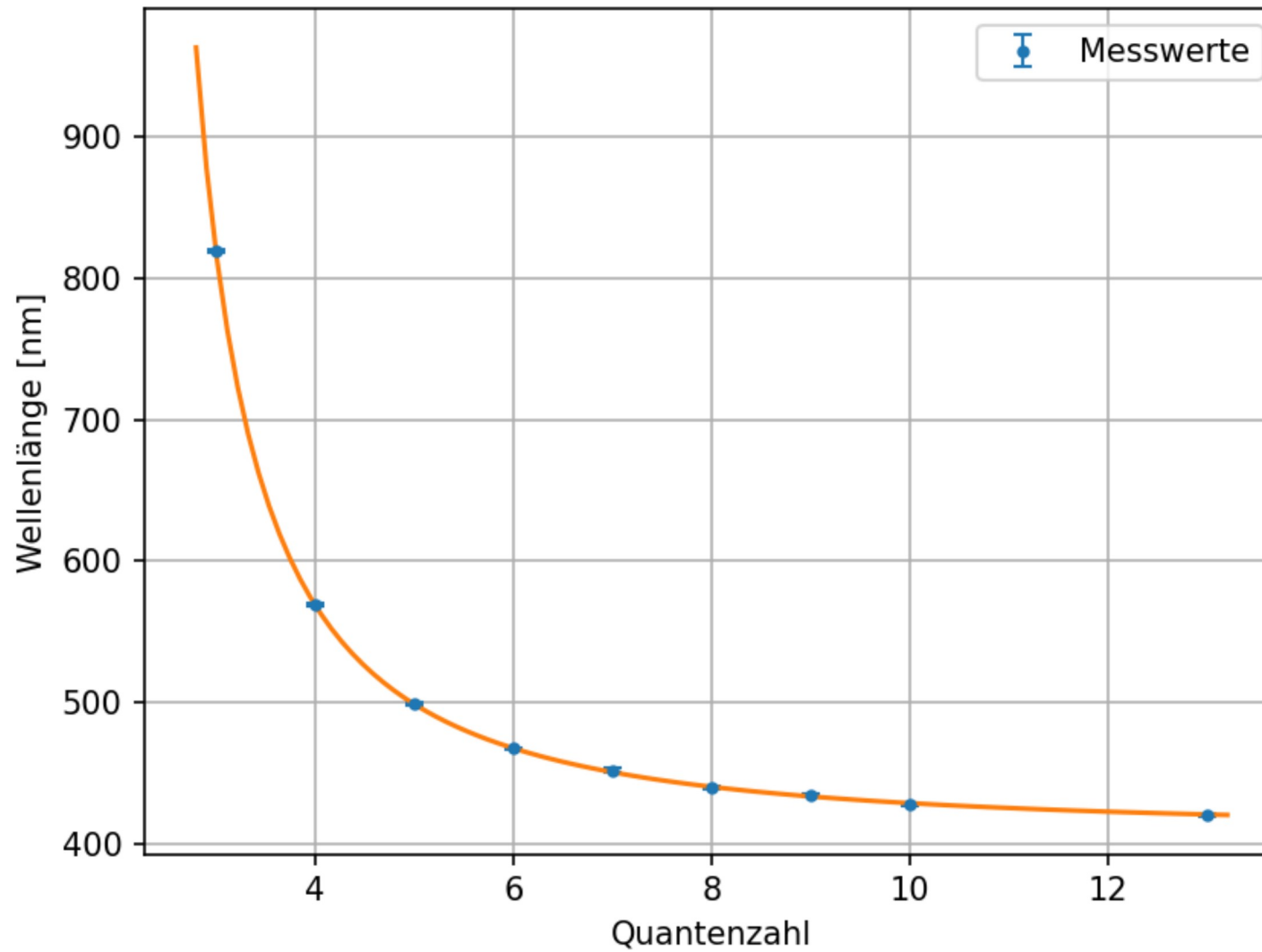
C:\Users\samue\AppData\Local\Temp\ipykernel_21880\988458060.py:7: RuntimeWarning: More than 20 figures have been opened. Figures created through the pyplot interface (`matplotlib.pyplot.figure`) are retained until explicitly closed and may consume too much memory. (To control this warning, see the rcParam `figure.max_open_warning`). Consider using `matplotlib.pyplot.close()`.

```

plt.figure(dpi=150)
E_Ry= -13.312622836061621 , Standardfehler= 0.2309511947591827
E_3p= -3.023694832517033 , Standardfehler= 0.003775784601552789
D_d= 0.03088361539955528 , Standardfehler= 0.02276220634648546
chi2= 2.5349938374137855
chi2_red= 0.4224989729022976
Wahrscheinlichkeit: 86.0 %

```

1. Nebenserie des Na-Atoms



Fitten der zweiten Nebenserie

```
In [30]: def fit_func2(m,E_Ry,E_3p,korrs):
          return 1.2398E3/(E_Ry/(m-korrs)**2-E_3p)
          para = [-13.6,-3,-0.02]
          popt, pcov = curve_fit(fit_func2,Nebenserie2m,Nebenserie2_wellen,sigma=Nebenserie2_wellenfehler,p0=para)
          x=np.linspace(4.8,13.2, 100)

          plt.figure(dpi=150)
```

```

plt.errorbar(Nebenserie2m,Nebenserie2_wellen,yerr=Nebenserie2_wellenfehler,fmt=".",capsize=3,elinewidth=1,label='Messwerte')
plt.plot(x,fit_func1(x,*popt))
plt.xlabel('Quantenzahl')
plt.ylabel('Wellenlänge [nm]')
plt.title('2. Nebenserie des Na-Atoms')
plt.legend()
plt.grid()
plt.savefig(Ausgabe_path/"Nr.3_Nebenserie2.png", format="png")

print("E_Ry=",popt[0], ", Standardfehler=", np.sqrt(pcov[0][0]))
print("E_3p=",popt[1], ", Standardfehler=", np.sqrt(pcov[1][1]))
print("D_s=",popt[2], ", Standardfehler=", np.sqrt(pcov[2][2]))

chi2_=np.sum((fit_func1(Nebenserie2m,*popt)- Nebenserie2_wellen)**2/ Nebenserie2_wellenfehler**2)
dof=len(Nebenserie2m)-3 #dof:degrees of freedom, Freiheitsgrad
chi2_red=chi2_/dof
print("chi2=", chi2_)
print("chi2_red=",chi2_red)

prob=round(1-chi2_.cdf(chi2_,dof),2)*100
print("Wahrscheinlichkeit:", prob,"%")

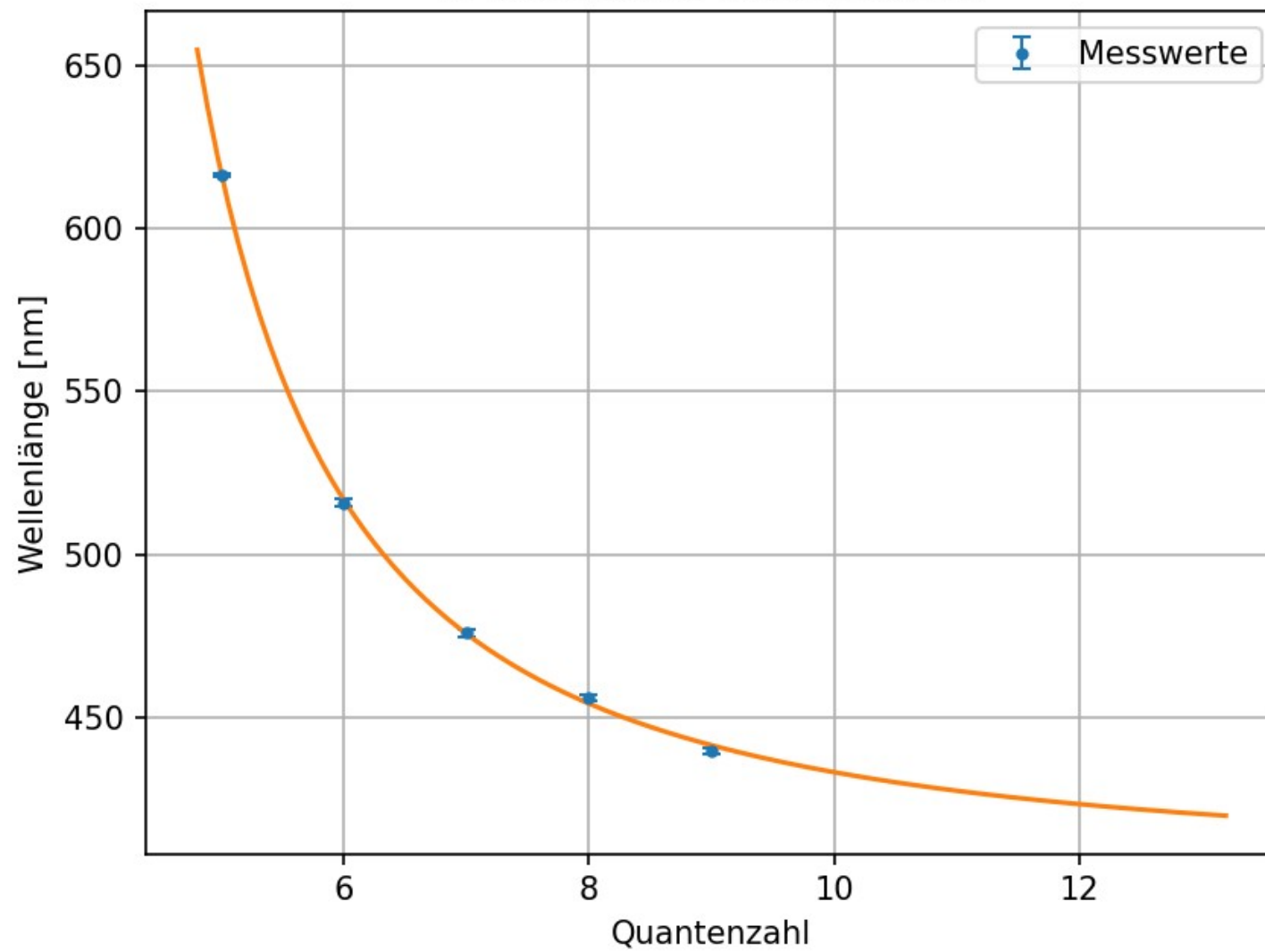
```

```

E_Ry= -15.424418960685733 , Standardfehler= 2.491190299549173
E_3p= -3.0596767847265967 , Standardfehler= 0.03237167290981489
D_s= 1.1628769594041732 , Standardfehler= 0.25291932741264594
chi2= 6.302600904126201
chi2_red= 3.1513004520631007
Wahrscheinlichkeit: 4.0 %

```

2. Nebenserie des Na-Atoms



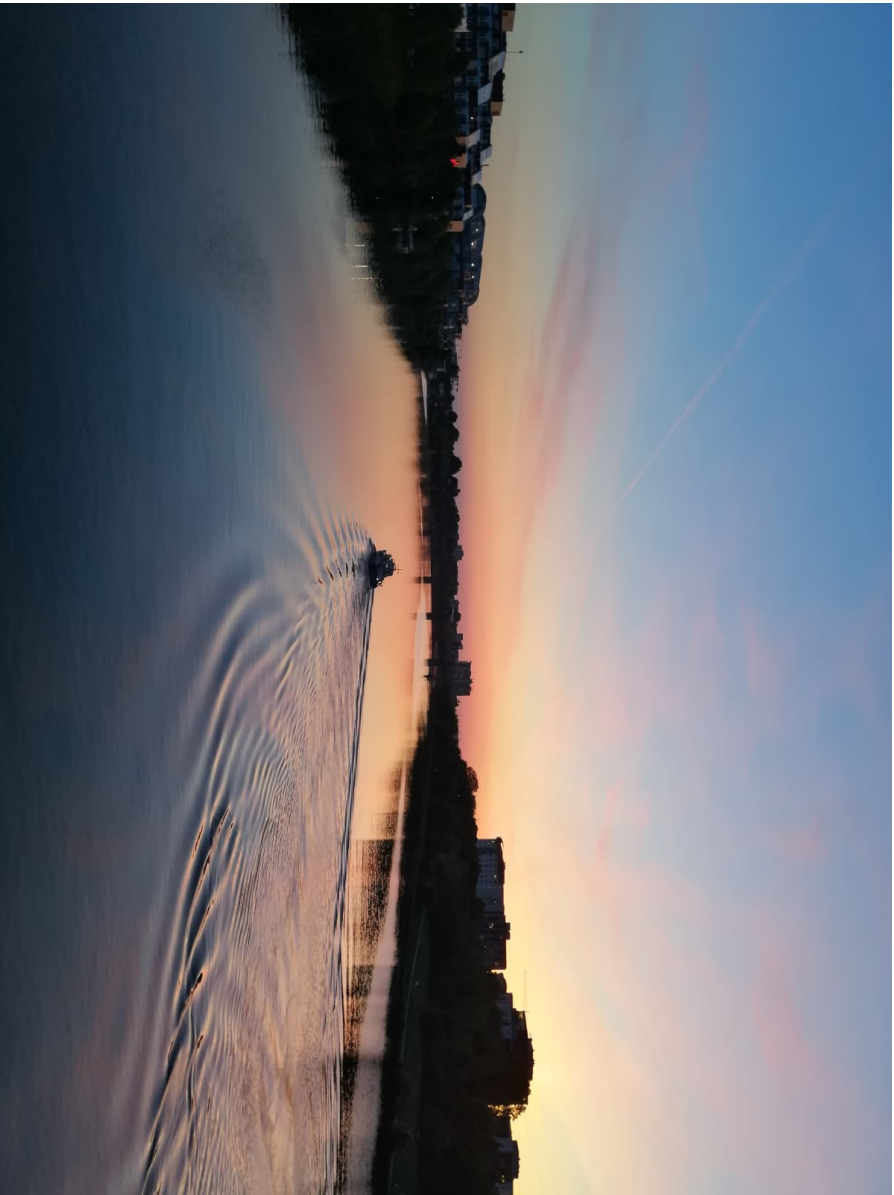


Abb. A.1: Sonnenuntergang mit Polizeiboot auf dem Weg von der Uni zum Bahnhof